

Expert C++, Parallel Computing, CUDA, and Real-Time Systems

GPU Programming, Signal Processing, Real-Time Architecture, and Heterogeneous Computing

Dr. Charles Dorner

C++ From Beginner to Expert — Book VI

First edition · 2026

About this expanded volume

This volume moves from parallel-computing fundamentals into CUDA C++, GPU memory behavior, asynchronous execution, profiling, signal processing, real-time design, and production heterogeneous systems. It uses the same operating discipline as the preceding volumes: establish a contract, trace the mechanism, implement a bounded change, verify correctness independently, measure the relevant performance quantity, and preserve a fallback path.

CUDA examples are instructional source. A CUDA compiler and supported NVIDIA GPU are required to validate device execution. CPU reference implementations are intentionally retained so numerical and architectural reasoning does not depend on access to one device.

How to use this book

Read each chapter in three passes. First identify the invariant and data path. Second trace the worked implementation and predict its failure modes before running it. Third run the lab on hardware you control, record compiler and device information, and compare observations with the stated acceptance criteria. Performance numbers in the text are examples of methodology, not promises for a particular GPU.

Chapter 1 — Understanding Parallel Computing

This chapter develops **understanding parallel computing** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

1.1 Identifying Parallel Work

1.1.1 Distinguishing Task and Data Parallelism

The engineering question in **Distinguishing Task and Data Parallelism** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Parallel software exposes dependencies that sequential execution can hide. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

1.1.2 Identifying Dependencies Between Operations

The engineering question in **Identifying Dependencies Between Operations** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Parallel software exposes dependencies that sequential execution can hide. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Independent partitions followed by a dependent reduction

Square two disjoint portions of an input concurrently, then sum only after both tasks have completed.

main.cpp

```
#include <array>
#include <future>
#include <numeric>
#include <iostream>

int main() {
    std::array<int,6> input{1,2,3,4,5,6}, squared{};
    auto transform=[&](std::size_t begin,std::size_t end) {
        for(auto i=begin;i<end;++i) squared[i]=input[i]*input[i];
    };
    auto first=std::async(std::launch::async,transform,0,3);
    auto second=std::async(std::launch::async,transform,3,6);
    first.get(); second.get();
    std::cout << "squares=";
    for(auto x:squared) std::cout << x << ' ';
    std::cout << "sum=" << std::accumulate(squared.begin(),squared.end(),0)
    << '\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Each task runs the same transformation over a separate index interval: this is data parallelism. Distinct array elements may be written concurrently. Both tasks borrow arrays whose lifetime extends until the futures have been joined.

The reduction depends on all six squares. Calling both `get()` operations before `accumulate` establishes the required completion boundary and propagates exceptions. Running the reduction concurrently with writes would introduce a data race, not additional valid parallelism.

Expected behavior or output

```
squares=1 4 9 16 25 36 sum=91
```

Concrete experiment

Change the partition boundary from three to four in both task calls. The output must remain identical; every index still needs exactly one writer.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

1.2 Understanding Limits to Parallel Speedup

1.2.1 Understanding Serial Bottlenecks

The engineering question in **Understanding Serial Bottlenecks** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Parallel software exposes dependencies that sequential execution can hide. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

1.2.2 Understanding Communication Overhead

The engineering question in **Understanding Communication Overhead** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Parallel software exposes dependencies that sequential execution can hide. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Compute the speedup ceiling including coordination cost

Evaluate Amdahl's model with a 20% serial fraction and an explicit overhead term; these numbers are assumptions, not benchmark results.

main.cpp

```
#include <array>
#include <iomanip>
#include <iostream>

int main() {
    constexpr double serial=0.20, overhead=0.04;
    std::cout << std::fixed << std::setprecision(3);
    for(int workers:std::array{1,2,4,8}) {
        const double seconds=serial+(1.0-serial)/workers+(workers>1?
overhead:0.0);
        std::cout << workers << " workers: " << 1.0/seconds << "x\n";
    }
    std::cout << "ideal infinite-worker ceiling=" << 1.0/serial << "x\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

A one-second baseline is split into `serial` and parallelizable work. Only $(1 - \text{serial})$ is divided by the worker count. The conditional overhead prevents charging parallel setup to the serial baseline.

At eight workers the modeled runtime is 0.34 seconds, not 0.125 seconds. Even infinitely many workers cannot eliminate the serial 0.20 seconds. The printed 5x ceiling deliberately excludes overhead; retaining 0.04 seconds lowers the asymptotic ceiling to about 4.167x.

Expected behavior or output

```
1 workers: 1.000x
2 workers: 1.562x
```

```
4 workers: 2.273x
8 workers: 2.941x
ideal infinite-worker ceiling=5.000x
```

Concrete experiment

Set overhead to 0.50. Predict which worker counts lose to the serial baseline, then compare the table. This models the danger of communication outweighing parallel work.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Evidence-driven understanding parallel computing

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path
```



```

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only `observed` with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result

transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth,

completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?

9. What hardware and software configuration produced the measurement?

10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should

therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 2 — Comparing CPU and GPU Architectures

This chapter develops **comparing cpu and gpu architectures** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

2.1 Understanding CPU Optimization Goals

2.1.1 Optimizing Low-Latency Sequential Execution

The engineering question in **Optimizing Low-Latency Sequential Execution** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. CPU and GPU processors optimize different mixtures of latency, throughput, and locality. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

2.1.2 Exploiting Large Cache Hierarchies

The engineering question in **Exploiting Large Cache Hierarchies** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. CPU and GPU processors optimize different mixtures of latency, throughput, and locality. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Trace dependent loads and contiguous work separately

Compare two access patterns that have different opportunities for CPU execution overlap without claiming measured cycle counts.

main.cpp

```
#include <array>
#include <iostream>
#include <numeric>

int main() {
    const std::array<std::size_t,6> next{3,5,4,1,0,2};
    std::size_t index=0;
    std::cout << "dependent path:";
    for(int step=0;step<6;++step) { std::cout << ' ' << index;
index=next[index]; }
    const std::array<int,6> contiguous{10,20,30,40,50,60};
    std::cout << "\ncontiguous sum="
                << std::accumulate(contiguous.begin(),contiguous.end(),0) <<
'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The next address in the first loop is determined by the value just loaded: `index=next[index]`. A CPU cannot know that dependency's result merely by issuing more iterations. The six-element permutation makes the dependency visible independently of real cache timing.

The second loop visits adjacent elements whose addresses are known from the iteration index. That pattern offers locality and potential unrolling/vectorization. Both arrays fit in a tiny working set here; the example demonstrates address dependencies, not a cache benchmark.

Expected behavior or output

```
dependent path: 0 3 1 5 2 4  
contiguous sum=210
```

Concrete experiment

Replace next with {1, 2, 3, 4, 5, 0}. The dependent addresses become contiguous but still depend on prior loads; explain why locality and dependency are different concerns.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

2.2 Understanding GPU Optimization Goals

2.2.1 Optimizing Massive Throughput

The engineering question in **Optimizing Massive Throughput** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. CPU and GPU processors optimize different mixtures of latency, throughput, and locality. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

2.2.2 Hiding Latency with Many Concurrent Threads

The engineering question in **Hiding Latency with Many Concurrent Threads** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. CPU and GPU processors optimize different mixtures of latency, throughput, and locality. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating

memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Scale a grid-stride throughput kernel

Assign independent array elements to many CUDA threads while retaining a host-side oracle for every result.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" ": "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
```

```

    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
{CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void ramp(float* out,std::size_t n) {
    for(std::size_t i=std::size_t(blockIdx.x)*blockDim.x+threadIdx.x;
        i<n;i+=std::size_t(blockDim.x)*gridDim.x)
        out[i]=float(i)*0.25f;
}

int main() {
    try {
        constexpr std::size_t n=100003;
        Device<float> out(n); std::vector<float> host(n);
        ramp<<<80,256>>>(out.p,n); CU(cudaGetLastError());

        CU(cudaMemcpy(host.data(),out.p,n*sizeof(float),cudaMemcpyDeviceToHost));
        for(std::size_t i=0;i<n;++i) require(host[i]==float(i)*0.25f,"ramp
mismatch");
        std::cout << "verified 100003 independent outputs\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

The initial index combines block and thread identity. The loop stride equals the total launched thread count, so a bounded grid covers inputs larger than itself. Each output location has one writer and no inter-thread dependency.

Many eligible warps give the scheduler alternative work when another warp waits. The program does not prove latency hiding or an optimal grid size:

occupancy, resource use, and actual profiling are still required. The blocking copy is the result-completion boundary.

Expected behavior or output

```
verified 100003 independent outputs
```

Concrete experiment

Change the grid from 80 to 4 blocks without changing `n`. All outputs must still match. Profile both only after correctness passes; fewer blocks can reduce available parallelism.

Verification status: gcc: target required

Technical references: [CUDA-GUIDE](#), [CUDA-MEMORY](#). See the added references and revision index.

Worked example — Evidence-driven comparing cpu and gpu architectures

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
```

```

const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
const auto expected = reference_transform(input);
const auto observed = reference_transform(input); // replace with
optimized path

if (expected.size() != observed.size()) return 2;
for (std::size_t i = 0; i < expected.size(); ++i) {
    if (!close_enough(expected[i], observed[i])) return 3;
}
std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only observed with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding

initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?

6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 3 — Building CUDA C++ Programs

This chapter develops **building cuda c++ programs** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

3.1 Understanding the CUDA Programming Model

3.1.1 Distinguishing Host and Device Code

The engineering question in **Distinguishing Host and Device Code** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. CUDA adds an explicit host/device execution boundary that must remain visible in design. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The

corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

3.1.2 Compiling CUDA Source with NVCC

The engineering question in **Compiling CUDA Source with NVCC** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. CUDA adds an explicit host/device execution boundary that must remain visible in design. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Compile one translation unit containing host and device functions

Use CUDA qualifiers to distinguish a reusable arithmetic function from the kernel entry and host orchestration.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
```

```

{CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
~Pinned(){(void)cudaFreeHost(p);}
Pinned(const Pinned&)=delete;
Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__host__ __device__ int affine(int x) { return 3*x+2; }
__global__ void evaluate(int* result) { result[0]=affine(7); }

int main() {
    try {
        Device<int> result(1); int observed=0;
        evaluate<<<1,1>>>(result.p); CU(cudaGetLastError());
        CU(cudaMemcpy(&observed,result.p,sizeof
observed,cudaMemcpyDeviceToHost));
        require(observed==affine(7),"host/device disagreement");
        std::cout << "host=" << affine(7) << " device=" << observed << '\n';
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

`affine` is compiled for both execution domains because it uses only operations valid in both. `evaluate` is a device kernel callable through launch syntax; its `int*` refers to device memory allocated by the host.

`nvcc` processes the `.cu` translation unit and coordinates host and device compilation. Ordinary `g++ main.cu` is not an equivalent build. The host invokes `affine` directly and compares its result with the independently executed kernel result.

Expected behavior or output

```

host=23 device=23

```

Concrete experiment

Change affine to $x \cdot x + 2$. Both paths should return 51 for input 7. Then remove `__device__` and observe the CUDA compilation error at the device call; retain the original before proceeding.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

3.2 Creating the First GPU Program

3.2.1 Allocating Device Memory

The engineering question in **Allocating Device Memory** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. CUDA adds an explicit host/device execution boundary that must remain visible in design. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

3.2.2 Launching a CUDA Kernel

The engineering question in **Launching a CUDA Kernel** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. CUDA adds an explicit host/device execution boundary that must remain visible in design. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or

fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Allocate two inputs, launch addition, and copy the result

Perform a complete CUDA allocation–transfer–launch–readback cycle with four checked integers.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p), n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
    {CU(cudaMallocHost(reinterpret_cast<void*>(&p), n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
```

```

        Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void add4(const int* a,const int* b,int* c) {
    unsigned i=threadIdx.x;
    if(i<4) c[i]=a[i]+b[i];
}

int main() {
    try {
        int a[4]{1,2,3,4}, b[4]{10,20,30,40}, c[4]{};
        Device<int> da(4),db(4),dc(4);
        CU(cudaMemcpy(da.p,a,sizeof a,cudaMemcpyHostToDevice));
        CU(cudaMemcpy(db.p,b,sizeof b,cudaMemcpyHostToDevice));
        add4<<<1,32>>>(da.p,db.p,dc.p); CU(cudaGetLastError());
        CU(cudaMemcpy(c,dc.p,sizeof c,cudaMemcpyDeviceToHost));
        for(int i=0;i<4;++i) require(c[i]==a[i]+b[i],"addition mismatch");
        std::cout << c[0] << ' ' << c[1] << ' ' << c[2] << ' ' << c[3] <<
'\n';
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

Three `Device<int>` owners each allocate storage for four integers and release it on scope exit. `sizeof a` is the byte count of the host array, not the number of elements. Copy direction is explicit for every transfer.

Only lanes0–3 write results; the remaining launched threads fail the `i<4` guard. `cudaGetLastError` checks launch setup, while the device-to-host copy also waits for the preceding default-stream computation before the CPU validates results.

Expected behavior or output

```

11 22 33 44

```

Concrete experiment

Change the fourth input to -4. The last result must become 36. Deliberately omitting an input copy is not a valid experiment to time; device allocation does not initialize input values.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

Worked example — Evidence-driven building cuda c++ programs

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
}
```

```
    }  
    std::cout << "verified " << observed.size() << " values\n";  
}
```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only `observed` with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A

number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?

10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 4 — Understanding CUDA

Threads, Blocks, and Grids

This chapter develops **understanding cuda threads, blocks, and grids** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

4.1 Mapping Work to Threads

4.1.1 Calculating Global Thread Indices

The engineering question in **Calculating Global Thread Indices** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Thread, block, and grid geometry is an indexing contract as much as a launch configuration. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

4.1.2 Preventing Out-of-Range Access

The engineering question in **Preventing Out-of-Range Access** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Thread, block, and grid geometry is an indexing contract as much as a launch configuration. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Guard a partially occupied final block

Write global indices for 259 elements using 128-thread blocks and verify a canary beyond the logical output.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p), n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
```

```

};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
{CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void index_values(int* out,unsigned n) {
    unsigned i=blockIdx.x*blockDim.x+threadIdx.x;
    if(i<n) out[i]=int(i);
}

int main() {
    try {
        constexpr unsigned n=259, block=128;
        std::vector<int> host(n+1,-99); Device<int> data(n+1);

        CU(cudaMemcpy(data.p,host.data(),host.size()*sizeof(int),cudaMemcpyHostToDevice));
        index_values<<<(n+block-1)/block,block>>>(data.p,n);
        CU(cudaGetLastError());

        CU(cudaMemcpy(host.data(),data.p,host.size()*sizeof(int),cudaMemcpyDeviceToHost));
        for(unsigned i=0;i<n;++i) require(host[i]==int(i),"index mismatch");
        require(host[n]==-99,"logical-end canary changed");
        std::cout << "last=258 canary=-99\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

Ceiling division launches three blocks, giving 384 physical threads for 259 logical elements. The guard is necessary because the last 125 threads otherwise write beyond the valid logical range.

The extra allocated element is a canary: it detects a write at index 259 even though that write would still be inside this allocation. A canary complements rather than replaces a memory checker; it does not cover every possible erroneous address.

Expected behavior or output

```
last=258 canary=-99
```

Concrete experiment

Set `n` to 256 and then 257. Predict the grid sizes 2 and 3. Keep the canary assertion in both cases to verify the boundary rather than just the first output.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

4.2 Organizing Threads into Blocks and Grids

4.2.1 Selecting Block Dimensions

The engineering question in **Selecting Block Dimensions** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Thread, block, and grid geometry is an indexing contract as much as a launch configuration. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

4.2.2 Scaling Kernels Across Large Inputs

The engineering question in **Scaling Kernels Across Large Inputs** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Thread, block, and grid geometry is an indexing contract as much as a launch configuration. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Map a rectangular image onto a two-dimensional grid

Cover a 19-by-11 row-major image with 8-by-4 thread blocks.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
```

```

    T* p=nullptr;
    explicit Pinned(std::size_t n)
    {CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void coordinates(int* image,unsigned width,unsigned height) {
    unsigned x=blockIdx.x*blockDim.x+threadIdx.x;
    unsigned y=blockIdx.y*blockDim.y+threadIdx.y;
    if(x<width && y<height) image[y*width+x]=int(100*y+x);
}

int main() {
    try {
        constexpr unsigned w=19,h=11;
        Device<int> image(w*h); std::vector<int> host(w*h);
        dim3 block(8,4),grid((w+7)/8,(h+3)/4);
        coordinates<<<grid,block>>>(image.p,w,h); CU(cudaGetLastError());

        CU(cudaMemcpy(host.data(),image.p,host.size()*sizeof(int),cudaMemcpyDeviceToHost));
        for(unsigned y=0;y<h;++y) for(unsigned x=0;x<w;++x)
            require(host[y*w+x]==int(100*y+x),"coordinate mismatch");
        std::cout << "grid=3x3 bottom-right=" << host.back() << '\n';
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

dim3 describes geometry, but device memory remains a flat allocation. $y \cdot \text{width} + x$ is the row-major storage index; the value $100 \cdot y + x$ independently encodes the expected coordinate for validation.

Width and height each need their own ceiling division and bounds check. A valid x coordinate does not imply a valid y coordinate. Block shape changes how adjacent threads map onto memory and must respect device thread/block limits.

Expected behavior or output

```
grid=3x3 bottom-right=1018
```

Concrete experiment

Use `block(16,2)` and recalculate both grid dimensions from the block object instead of hardcoded 7 and 3. Output values must remain unchanged although the launch geometry changes.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

Worked example — Evidence-driven understanding cuda threads, blocks, and grids

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
```

optimized path

```
    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}
```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only observed with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful

trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?

7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 5 — Understanding Warps and GPU Execution

This chapter develops **understanding warps and gpu execution** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

5.1 Understanding Warp-Based Execution

5.1.1 Grouping Threads into Warps

The engineering question in **Grouping Threads into Warps** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Warp execution turns control-flow shape into a throughput concern. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating

memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

5.1.2 Understanding SIMT Execution

The engineering question in **Understanding SIMT Execution** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Warp execution turns control-flow shape into a throughput concern. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Observe warp lanes and a converged ballot

Use one full32-thread warp to report which lanes satisfy a predicate; no device printf ordering is assumed.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
```

```

{CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
~Pinned(){(void)cudaFreeHost(p);}
Pinned(const Pinned&)=delete;
Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void ballot_demo(unsigned* masks) {
    unsigned lane=threadIdx.x*warpSize;
    unsigned mask=__ballot_sync(0xffffffffu,lane<4);
    masks[threadIdx.x]=mask;
}

int main() {
    try {
        cudaDeviceProp prop{}; CU(cudaGetDeviceProperties(&prop,0));
        require(prop.warpSize==32,"this ballot fixture requires 32-lane
warps");
        Device<unsigned> data(32); std::vector<unsigned> host(32);
        ballot_demo<<<1,32>>>(data.p); CU(cudaGetLastError());

        CU(cudaMemcpy(host.data(),data.p,32*sizeof(unsigned),cudaMemcpyDeviceToHost))
        ;
        for(auto mask:host) require(mask==15u,"ballot mismatch");
        std::cout << "32 lanes observed mask=15\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

All 32 launched lanes execute the same ballot call with a full participation mask. The predicate is true for lanes 0–3, so bits 0–3 are set and the unsigned result is 15 in every lane.

The mask is an explicit collective-participation promise, not a way to make absent threads appear. This example deliberately avoids partial warps and divergent placement of the collective. More general kernels must derive a mask appropriate to their actual participants.

Expected behavior or output

```
32 lanes observed mask=15
```

Concrete experiment

Change the predicate to `lane%2==0`. Predict `0x55555555`, then change the host assertion and display format. Do not move the ballot into a branch executed by only some named lanes.

Verification status: gcc: target required

Technical references: [CUDA-GUIDE](#), [CUDA-MEMORY](#). See the added references and revision index.

5.2 Understanding Branch Divergence

5.2.1 Recognizing Divergent Control Flow

The engineering question in **Recognizing Divergent Control Flow** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Warp execution turns control-flow shape into a throughput concern. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

5.2.2 Restructuring Kernels to Reduce Divergence

The engineering question in **Restructuring Kernels to Reduce Divergence** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Warp execution turns control-flow shape into a throughput concern. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The

corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Compare alternating and grouped branch predicates

Produce the same piecewise transform with two physical work orderings and restore results to their original logical indices.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e != cudaSuccess) throw std::runtime_error(
        std::string(operation) + ": " + cudaGetErrorString(e));
}
#define CU(call) check((call), #call)
template<class T> struct Device {
    T* p = nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p), n * sizeof(T)));
    }
    ~Device() { if(p) (void)cudaFree(p); }
    Device(const Device&) = delete;
    Device& operator=(const Device&) = delete;
};
struct Stream {
    cudaStream_t s{};
    Stream() { CU(cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking)); }
    ~Stream() { (void)cudaStreamSynchronize(s); (void)cudaStreamDestroy(s); }
    Stream(const Stream&) = delete;
    Stream& operator=(const Stream&) = delete;
};
struct Event {
    cudaEvent_t e{};
    Event() { CU(cudaEventCreate(&e)); }
    ~Event() { (void)cudaEventDestroy(e); }
    Event(const Event&) = delete;
```

```

    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
    {CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void piecewise(float* out,bool grouped) {
    unsigned lane=threadIdx.x;
    unsigned logical=grouped ? (lane<16?2*lane:2*(lane-16)+1) : lane;
    float x=float(logical);
    out[logical]=(logical%2==0)?x*x:-x;
}

int main() {
    try {
        Device<float> a(32),b(32); std::vector<float> ha(32),hb(32);
        piecewise<<<1,32>>>(a.p,false); CU(cudaGetLastError());
        piecewise<<<1,32>>>(b.p,true); CU(cudaGetLastError());
        CU(cudaMemcpy(ha.data(),a.p,128,cudaMemcpyDeviceToHost));
        CU(cudaMemcpy(hb.data(),b.p,128,cudaMemcpyDeviceToHost));
        require(ha==hb,"reordering changed logical values");
        std::cout << "same values: even30=" << ha[30] << " odd31=" << ha[31]
        << '\n';
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

The ungrouped mapping alternates branch outcomes across lanes. The grouped mapping assigns even logical indices to the first half-warp and odd indices to the second. Stores use logical indices, so work reordering does not reorder the externally visible result.

Grouping within one warp still leaves two outcomes; it does not eliminate divergence completely. With larger batches, grouping whole warps can do so, but may worsen memory coalescing or require a permutation pass. The compiler may

also use predication for this small expression; inspect generated code before claiming a speedup.

Expected behavior or output

```
same values: even30=900 odd31=-31
```

Concrete experiment

Extend the fixture to 64 elements and group all 32 even indices into one warp. Verify the inverse mapping and profile permutation cost separately from kernel time.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

Worked example — Evidence-driven understanding warps and gpu execution

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
```

```

const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
const auto expected = reference_transform(input);
const auto observed = reference_transform(input); // replace with
optimized path

if (expected.size() != observed.size()) return 2;
for (std::size_t i = 0; i < expected.size(); ++i) {
    if (!close_enough(expected[i], observed[i])) return 3;
}
std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only observed with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding

initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?

6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 6 — Designing Efficient CUDA Kernels

This chapter develops **designing efficient cuda kernels** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

6.1 Mapping Algorithms to GPU Execution

6.1.1 Identifying Independent Operations

The engineering question in **Identifying Independent Operations** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Kernel design should expose enough independent work without obscuring boundary conditions. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

6.1.2 Selecting Appropriate Parallel Granularity

The engineering question in **Selecting Appropriate Parallel Granularity** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Kernel design should expose enough independent work without obscuring boundary conditions. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Assign one independent row dot product to each thread

Map a5-by3 matrix-vector multiplication to five independent outputs rather than launching a thread for every multiply without a reduction design.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p), n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
```

```

};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
{CU(cudaMallocHost(reinterpret_cast<void**>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void matvec(const float* a,const float* x,float* y,unsigned
rows,unsigned cols) {
    unsigned row=blockIdx.x*blockDim.x+threadIdx.x;
    if(row>=rows) return;
    float sum=0;
    for(unsigned col=0;col<cols;++col) sum+=a[row*cols+col]*x[col];
    y[row]=sum;
}

int main() {
    try {
        float a[15]{1,2,3, 4,5,6, 7,8,9, 10,11,12, 13,14,15};
        float x[3]{1,0,-1},y[5]{};
        Device<float> da(15),dx(3),dy(5);
        CU(cudaMemcpy(da.p,a,sizeof a,cudaMemcpyHostToDevice));
        CU(cudaMemcpy(dx.p,x,sizeof x,cudaMemcpyHostToDevice));
        matvec<<<1,32>>>(da.p,dx.p,dy.p,5,3); CU(cudaGetLastError());
        CU(cudaMemcpy(y,dy.p,sizeof y,cudaMemcpyDeviceToHost));
        for(auto v:y) require(v==-2.0f,"dot product mismatch");
        std::cout << "five row dot products=-2\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

Rows are independent, whereas the three products within one row contribute to one reduction. A thread owns its scalar sum and writes exactly one `y[ row]`, avoiding atomics and cross-thread reduction machinery.

This granularity is intentionally simple for short rows. Long rows may need cooperative reduction; tiny five-row inputs do not justify GPU offload for

performance. The example demonstrates a correct work decomposition rather than an efficient production matrix library.

Expected behavior or output

```
five row dot products=-2
```

Concrete experiment

Change x to $\{1,1,1\}$; the outputs should be 6,15,24,33,42. Then reason how row count and column count affect parallelism independently.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

6.2 Managing Kernel Boundaries

6.2.1 Handling Arbitrary Input Sizes

The engineering question in **Handling Arbitrary Input Sizes** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Kernel design should expose enough independent work without obscuring boundary conditions. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements

should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

6.2.2 Minimizing Launch Overhead

The engineering question in **Minimizing Launch Overhead** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Kernel design should expose enough independent work without obscuring boundary conditions. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Fuse two elementwise passes without changing semantics

Compare two kernels with a temporary array against one fused kernel on 513 elements.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
```

```

};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
    {CU(cudaMallocHost(reinterpret_cast<void**>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void twice(const float* x,float* t,unsigned n) {
    unsigned i=blockIdx.x*blockDim.x+threadIdx.x;if(i<n)t[i]=2*x[i];
}
__global__ void offset(const float* t,float* y,unsigned n) {
    unsigned i=blockIdx.x*blockDim.x+threadIdx.x;if(i<n)y[i]=t[i]+3;
}
__global__ void fused(const float* x,float* y,unsigned n) {
    unsigned i=blockIdx.x*blockDim.x+threadIdx.x;if(i<n)y[i]=2*x[i]+3;
}

int main() {
    try {
        constexpr unsigned n=513; std::vector<float> x(n),a(n),b(n);
        for(unsigned i=0;i<n;++i)x[i]=float(i);
        Device<float> dx(n),temp(n),da(n),db(n);
        CU(cudaMemcpy(dx.p,x.data(),n*sizeof(float),cudaMemcpyHostToDevice));
        twice<<<3,256>>>>(dx.p,temp.p,n); CU(cudaGetLastError());
        offset<<<3,256>>>>(temp.p,da.p,n); CU(cudaGetLastError());
        fused<<<3,256>>>>(dx.p,db.p,n); CU(cudaGetLastError());
        CU(cudaMemcpy(a.data(),da.p,n*sizeof(float),cudaMemcpyDeviceToHost));
        CU(cudaMemcpy(b.data(),db.p,n*sizeof(float),cudaMemcpyDeviceToHost));
        require(a==b && b.back()==1027.0f,"fusion mismatch");
        std::cout << "513 equal outputs, last=1027\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

The split path materializes `temp`: one kernel writes it and another reads it. Fusion eliminates that intermediate traffic and one launch, while retaining the guard needed for an arbitrary input length.

Default-stream ordering makes the split dependency correct without a host wait between launches. These particular small integer-valued floats are exactly representable; more complex fusion can change floating-point association or FMA behavior and requires tolerance-based validation.

Expected behavior or output

```
513 equal outputs, last=1027
```

Concrete experiment

Use input lengths 1,256,257. Recompute the grid with ceiling division instead of the fixture's fixed 3. Benchmark split and fused paths with identical transfer scopes.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

Worked example — Evidence-driven designing efficient cuda kernels

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}
```

```

}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only observed with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering,

and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?

3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling

instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 7 — Understanding the CUDA Memory Hierarchy

This chapter develops **understanding the cuda memory hierarchy** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

7.1 Working with Global Device Memory

7.1.1 Allocating and Freeing Device Buffers

The engineering question in **Allocating and Freeing Device Buffers** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. The memory hierarchy makes data movement a first-class part of algorithm design. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating

memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

7.1.2 Transferring Data Between Host and Device

The engineering question in **Transferring Data Between Host and Device** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. The memory hierarchy makes data movement a first-class part of algorithm design. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Copy and modify only a device subrange

Demonstrate that `cudaMemcpy` sizes are bytes and that a device pointer plus an element offset addresses a slice.

CUDA target example. No `nvcc` compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run. Expected display has one trailing space before its newline.

`cuda_support.hpp`

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e != cudaSuccess) throw std::runtime_error(
        std::string(operation) + ": " + cudaGetErrorString(e));
}
#define CU(call) check((call), #call)
template<class T> struct Device {
    T* p = nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p), n * sizeof(T)));
    }
    ~Device() { if(p) (void)cudaFree(p); }
    Device(const Device&) = delete;
    Device& operator=(const Device&) = delete;
};
struct Stream {
    cudaStream_t s{};
    Stream() { CU(cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking)); }
    ~Stream() { (void)cudaStreamSynchronize(s); (void)cudaStreamDestroy(s); }
    Stream(const Stream&) = delete;
    Stream& operator=(const Stream&) = delete;
};
struct Event {
    cudaEvent_t e{};
    Event() { CU(cudaEventCreate(&e)); }
    ~Event() { (void)cudaEventDestroy(e); }
    Event(const Event&) = delete;
```

```

        Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
    {CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void negate(int* values,unsigned n) {
    unsigned i=threadIdx.x;if(i<n)values[i]=-values[i];
}

int main() {
    try {
        int input[8]{0,1,2,3,4,5,6,7},output[8]{};
        Device<int> data(8);
        CU(cudaMemcpy(data.p,input,sizeof input,cudaMemcpyHostToDevice));
        negate<<<1,32>>>(data.p+2,3); CU(cudaGetLastError());
        CU(cudaMemcpy(output,data.p,sizeof output,cudaMemcpyDeviceToHost));
        int expected[8]{0,1,-2,-3,-4,5,6,7};
        for(unsigned i=0;i<8;++i)require(output[i]==expected[i],"slice
mismatch");
        for(auto x:output)std::cout << x << ' ';
        std::cout << '\n';
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

`data.p+2` advances by two integers, but the transfer sizes use `sizeof` in bytes. The kernel receives a view starting at original element2 and owns only three writes.

The Device object remains the allocation owner; passing an interior pointer does not transfer ownership or authorize freeing that pointer. The untouched prefix/suffix in the expected array detect an overbroad update.

Expected behavior or output

```
0 1 -2 -3 -4 5 6 7
```

Concrete experiment

Change the slice to `offset6,count2`. The last two values should be negated. Reject `offset+count` beyond 8 on the host before launching.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

7.2 Understanding Specialized GPU Memory

7.2.1 Understanding Registers and Local Memory

The engineering question in **Understanding Registers and Local Memory** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. The memory hierarchy makes data movement a first-class part of algorithm design. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

7.2.2 Understanding Constant and Texture Memory

The engineering question in **Understanding Constant and Texture Memory** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. The memory hierarchy makes data movement a first-class part of algorithm design. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating

memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Use constant coefficients and inspect per-thread storage

Apply a three-tap finite impulse response using a constant-memory coefficient array and thread-local accumulators.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" ": +cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
```

```

    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
{CU(cudaMallocHost(reinterpret_cast<void**>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__constant__ float taps[3];
__global__ void fir3(const float* input,float* output,unsigned n) {
    unsigned i=blockIdx.x*blockDim.x+threadIdx.x;
    if(i>=n)return;
    float sum=0;
    for(unsigned k=0;k<3;++k)if(i>=k)sum+=taps[k]*input[i-k];
    output[i]=sum;
}

int main() {
    try {
        float weights[3]{0.25f,0.5f,0.25f},input[5]{1,2,3,4,5},output[5]{};
        Device<float> x(5),y(5);
        CU(cudaMemcpyToSymbol(taps,weights,sizeof weights));
        CU(cudaMemcpy(x.p,input,sizeof input,cudaMemcpyHostToDevice));
        fir3<<<1,32>>>(x.p,y.p,5);CU(cudaGetLastError());
        CU(cudaMemcpy(output,y.p,sizeof output,cudaMemcpyDeviceToHost));
        float expected[5]{0.25f,1.0f,2.0f,3.0f,4.0f};
        for(int i=0;i<5;++i)require(output[i]==expected[i],"FIR mismatch");
        std::cout << "FIR: 0.25 1 2 3 4\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 -Xptxas=-v main.cu -o example
./example

```

Explanation

`cudaMemcpyToSymbol` initializes the device constant symbol before any kernel uses it. At each loop iteration active lanes read the same coefficient address, a pattern suitable for constant-memory broadcast. The input remains ordinary global memory.

sum, i, and k are per-thread values; the compiler decides register allocation and possible spills. Compile with `-Xptxas=-v` to inspect register/local-memory usage instead of assuming every local variable is a register. Texture objects serve different cached sampling/addressing needs and are not required for these uniform coefficients.

Expected behavior or output

```
FIR: 0.25 1 2 3 4
```

Concrete experiment

Change the taps to {1,-1,0}. Predict 1,1,1,1 for this ramp. Inspect compiler resource output before and after adding a large thread-local scratch array.

Verification status: gcc: target required

Technical references: [CUDA-GUIDE](#), [CUDA-MEMORY](#). See the added references and revision index.

Worked example — Evidence-driven understanding the cuda memory hierarchy

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}
```

```

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only observed with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding

initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?

6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 8 — Optimizing Kernels with Shared Memory

This chapter develops **optimizing kernels with shared memory** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

8.1 Using Programmer-Managed Shared Storage

8.1.1 Loading Reusable Data into Shared Memory

The engineering question in **Loading Reusable Data into Shared Memory** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Shared memory is useful only when reuse compensates for synchronization and staging cost. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

8.1.2 Synchronizing Threads Within Blocks

The engineering question in **Synchronizing Threads Within Blocks** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Shared memory is useful only when reuse compensates for synchronization and staging cost. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Reuse a shared tile across neighboring output samples

Stage a block's samples and one left halo so a two-point average can reuse data safely.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p), n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
```

```

};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
    {CU(cudaMallocHost(reinterpret_cast<void**>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void average2(const float* x,float* y,unsigned n) {
    __shared__ float tile[129];
    unsigned local=threadIdx.x, global=blockIdx.x*blockDim.x+local;
    tile[local+1]=global<n?x[global]:0;
    if(local==0)tile[0]=global>0?x[global-1]:0;
    __syncthreads();
    if(global<n)y[global]=0.5f*(tile[local]+tile[local+1]);
}

int main() {
    try {
        constexpr unsigned n=131;std::vector<float>x(n),y(n);
        for(unsigned i=0;i<n;++i)x[i]=float(i+1);
        Device<float> dx(n),dy(n);
        CU(cudaMemcpy(dx.p,x.data(),n*sizeof(float),cudaMemcpyHostToDevice));
        average2<<<2,128>>>(dx.p,dy.p,n);CU(cudaGetLastError());
        CU(cudaMemcpy(y.data(),dy.p,n*sizeof(float),cudaMemcpyDeviceToHost));
        for(unsigned i=0;i<n;++i)require(y[i]==0.5f*(x[i]+(i?x[i-1]:0)),"tile
mismatch");
        std::cout << "first=0.5 block-boundary=128.5 last=130.5\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

Each thread stages its input in `tile[local+1]`. Thread0 also supplies the preceding sample in `tile[0]`, including the halo across a block boundary. The extra element is necessary for the first output in every block.

Every thread reaches `__syncthreads`, including lanes beyond `n`; those lanes stage zero but do not write outputs. Returning before the barrier in only some threads

would violate the synchronization design. The static shared array assumes the documented 128-thread launch.

Expected behavior or output

```
first=0.5 block-boundary=128.5 last=130.5
```

Concrete experiment

Set $n=128$ and 129 , then check the first element in the second block. Explain why setting every block's left halo to zero would create a discontinuity.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

8.2 Avoiding Shared-Memory Bottlenecks

8.2.1 Understanding Bank Conflicts

The engineering question in **Understanding Bank Conflicts** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Shared memory is useful only when reuse compensates for synchronization and staging cost. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements

should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

8.2.2 Padding Shared Data Structures

The engineering question in **Padding Shared Data Structures** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Shared memory is useful only when reuse compensates for synchronization and staging cost. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Pad a transpose tile to change bank mapping

Transpose a 32-by-32 matrix with shared-memory leading dimensions 32 and 33; compare correctness before measuring conflicts.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
```

```

};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
{CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}
}

```

main.cu

```

#include "cuda_support.hpp"
template<int Pitch> __global__ void transpose(const float* x,float* y) {
    __shared__ float tile[32][Pitch];
    unsigned row=threadIdx.y,col=threadIdx.x;
    tile[row][col]=x[row*32+col];
    __syncthreads();
    y[row*32+col]=tile[col][row];
}

int main() {
    try {
        cudaDeviceProp prop{};CU(cudaGetDeviceProperties(&prop,0));
        require(prop.maxThreadsPerBlock>=1024,"fixture needs 1024-thread
blocks");
        std::vector<float>x(1024),a(1024),b(1024);
        for(unsigned i=0;i<x.size();++i)x[i]=float(i);
        Device<float> dx(1024),da(1024),db(1024);
        CU(cudaMemcpy(dx.p,x.data(),4096,cudaMemcpyHostToDevice));
        transpose<32><<<1,dim3(32,32)>>>(dx.p,da.p);CU(cudaGetLastError());
        transpose<33><<<1,dim3(32,32)>>>(dx.p,db.p);CU(cudaGetLastError());
        CU(cudaMemcpy(a.data(),da.p,4096,cudaMemcpyDeviceToHost));
        CU(cudaMemcpy(b.data(),db.p,4096,cudaMemcpyDeviceToHost));
        for(unsigned r=0;r<32;++r)for(unsigned c=0;c<32;++c)
            require(a[r*32+c]==x[c*32+r] && b[r*32+c]==a[r*32+c],"transpose
mismatch");
        std::cout << "both tile pitches transpose 1024 values\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```


Explanation

A warp varies `threadIdx.x` first. Reading `tile[col][row]` therefore visits a column. With 32-bit elements and a 32-bank mapping, a pitch 32 column maps lanes to the same bank; pitch 33 rotates successive rows through banks.

Padding changes addresses, not the mathematical transpose. It consumes extra shared memory and does not guarantee a speedup on every architecture or tiny workload. Inspect shared-memory conflict metrics on the target rather than inferring timing from equality of outputs.

Expected behavior or output

```
both tile pitches transpose 1024 values
```

Concrete experiment

Profile both kernels with Nsight Compute. Record shared-memory transactions/conflicts and runtime, including the device model. Keep the 1024-value correctness oracle.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, NSIGHT-COMPUTE. See the added references and revision index.

Worked example — Evidence-driven optimizing kernels with shared memory

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
}
```

```

    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only `observed` with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic

side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers

are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?

2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 9 — Optimizing Global Memory Access

This chapter develops **optimizing global memory access** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

9.1 Understanding Memory Coalescing

9.1.1 Aligning Adjacent Threads with Adjacent Data

The engineering question in **Aligning Adjacent Threads with Adjacent Data** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Coalescing depends on the relationship between thread indices and physical addresses. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

9.1.2 Recognizing Strided Access Penalties

The engineering question in **Recognizing Strided Access Penalties** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Coalescing depends on the relationship between thread indices and physical addresses. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Compare contiguous reads with a strided permutation

Read the same 4096 inputs through either adjacent addresses or a 32-way interleaving.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p), n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
```

```

};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
{CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void gather(const float* x,float* y,bool strided) {
    unsigned i=blockIdx.x*blockDim.x+threadIdx.x;
    if(i<4096){unsigned src=strided?(i%128)*32+i/128:i;y[i]=x[src];}
}

int main() {
    try {
        std::vector<float>x(4096),a(4096),b(4096);for(unsigned i=0;i<4096;+
+i)x[i]=float(i);
        Device<float> dx(4096),da(4096),db(4096);

        CU(cudaMemcpy(dx.p,x.data(),x.size()*sizeof(float),cudaMemcpyHostToDevice));
        gather<<<16,256>>>(dx.p,da.p,false);CU(cudaGetLastError());
        gather<<<16,256>>>(dx.p,db.p,true);CU(cudaGetLastError());

        CU(cudaMemcpy(a.data(),da.p,a.size()*sizeof(float),cudaMemcpyDeviceToHost));

        CU(cudaMemcpy(b.data(),db.p,b.size()*sizeof(float),cudaMemcpyDeviceToHost));
        for(unsigned i=0;i<4096;++i)require(a[i]==x[i]&&b[i]==x[(i%128)*32+i/
128],"gather mismatch");
        std::cout << "contiguous first=0 strided second=32\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

Neighboring lanes in the contiguous case read neighboring floats. In the strided case their source addresses differ by 32 floats, spreading a warp's requests across more memory regions. Output stores remain contiguous in both cases.

The index expression is a permutation of 0...4095, so both paths access the same allocation without out-of-range reads. Their output order intentionally differs and

each has its own oracle. Compare transaction efficiency, not just wall time from one small launch.

Expected behavior or output

```
contiguous first=0 strided second=32
```

Concrete experiment

Change the permutation to $(i\%64)*64+i/64$ and update the host oracle. Explain which part of the expression sets the adjacent-lane stride.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

9.2 Designing GPU-Friendly Data Layouts

9.2.1 Comparing Array-of-Structures Layouts

The engineering question in **Comparing Array-of-Structures Layouts** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Coalescing depends on the relationship between thread indices and physical addresses. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements

should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

9.2.2 Comparing Structure-of-Arrays Layouts

The engineering question in **Comparing Structure-of-Arrays Layouts** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Coalescing depends on the relationship between thread indices and physical addresses. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Extract mass from AoS and SoA representations

Run one mass-only transformation against two layouts with equivalent particle data.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
```

```

};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
    {CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}
}

```

main.cu

```

#include "cuda_support.hpp"
struct Particle {float x,y,z,mass;};
__global__ void aos(const Particle* p,float* y,unsigned n){unsigned
i=threadIdx.x;if(i<n)y[i]=2*p[i].mass;}
__global__ void soa(const float* mass,float* y,unsigned n){unsigned
i=threadIdx.x;if(i<n)y[i]=2*mass[i];}

int main() {
    try {
        Particle p[8]{};float mass[8]{};a[8]{};b[8]{};
        for(unsigned i=0;i<8;++i){p[i]={1,2,3,float(i+1)};mass[i]=p[i].mass;}
        Device<Particle> dp(8);Device<float> dm(8),da(8),db(8);
        CU(cudaMemcpy(dp.p,p,sizeof
p,cudaMemcpyHostToDevice));CU(cudaMemcpy(dm.p,mass,sizeof
mass,cudaMemcpyHostToDevice));

        aos<<<1,32>>>(dp.p,da.p,8);CU(cudaGetLastError());soa<<<1,32>>>(dm.p,db.p,8);
        CU(cudaGetLastError());
        CU(cudaMemcpy(a,da.p,sizeof
a,cudaMemcpyDeviceToHost));CU(cudaMemcpy(b,db.p,sizeof
b,cudaMemcpyDeviceToHost));
        for(unsigned i=0;i<8;++i)require(a[i]==2*mass[i]&&b[i]==a[i],"layout
mismatch");
        std::cout << "mass-only outputs 2 through16 agree\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

AoS places four fields in each particle; consecutive mass loads are separated by the struct stride. SoA stores only the masses next to one another, which suits this mass-only kernel. Both kernels implement the identical arithmetic.

Changing layout can add conversion cost or inconvenience workloads using all fields. The example separates that architectural tradeoff from correctness and makes no speed claim for eight particles. Keep host/device layout definitions identical when transferring whole structs.

Expected behavior or output

```
mass-only outputs 2 through 16 agree
```

Concrete experiment

Add an operation that uses `x,y,z, and mass`. Measure whether SoA is still advantageous for that access pattern, including any required layout conversion.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

Worked example — Evidence-driven optimizing global memory access

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}
```

```

}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only `observed` with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering,

and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?

3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling

instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 10 — Synchronizing Parallel GPU Work

This chapter develops **synchronizing parallel gpu work** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

10.1 Synchronizing Threads Within Blocks

10.1.1 Using Block-Level Barriers

The engineering question in **Using Block-Level Barriers** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Synchronization is a correctness operation before it is a performance cost. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating

memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

10.1.2 Preventing Shared-Memory Races

The engineering question in **Preventing Shared-Memory Races** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Synchronization is a correctness operation before it is a performance cost. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Reduce one block with uniform barrier participation

Sum64 integers through a shared-memory tree; active arithmetic lanes shrink while all threads continue to reach each barrier.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
```

```

{CU(cudaMallocHost(reinterpret_cast<void**>(&p),n*sizeof(T)));}
~Pinned(){(void)cudaFreeHost(p);}
Pinned(const Pinned&)=delete;
Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void reduce64(const int* input,int* output) {
    __shared__ int work[64];unsigned t=threadIdx.x;
    work[t]=input[t];__syncthreads();
    for(unsigned stride=32;stride;stride/=2){
        if(t<stride)work[t]+=work[t+stride];
        __syncthreads();
    }
    if(t==0)output[0]=work[0];
}

int main() {
    try {
        int x[64];for(int i=0;i<64;++i)x[i]=i+1;
        Device<int> dx(64),dy(1);int sum=0;
        CU(cudaMemcpy(dx.p,x,sizeof x,cudaMemcpyHostToDevice));
        reduce64<<<1,64>>>(dx.p,dy.p);CU(cudaGetLastError());
        CU(cudaMemcpy(&sum,dy.p,sizeof sum,cudaMemcpyDeviceToHost));
        require(sum==2080,"reduction mismatch");std::cout << "sum=" << sum <<
'\n';
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

At stride32, lanes0–31 combine two values; at stride16, lanes0–15 combine those partial sums. The barrier after every level makes previous writes visible before the next level reads them.

`__syncthreads` is outside the `if (t<stride)` block, so the full block participates even after some threads stop performing additions. The kernel's fixed64-element contract is deliberate; arbitrary sizes need padding or another reduction design.

Expected behavior or output

```
sum=2080
```

Concrete experiment

Use 64 ones and expect 64. Then describe why putting the barrier inside `if (t < stride)` is invalid rather than attempting to benchmark the broken version.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

10.2 Coordinating Across Kernels

10.2.1 Using Kernel Boundaries for Global Synchronization

The engineering question in **Using Kernel Boundaries for Global Synchronization** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Synchronization is a correctness operation before it is a performance cost. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

10.2.2 Understanding Device-Wide Synchronization

The engineering question in **Understanding Device-Wide Synchronization** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Synchronization is a correctness operation before it is a performance cost. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The

corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Use an event to order dependent kernels in different streams

Initialize data in one stream and consume it in another only after an explicit event dependency.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
```

```

    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
    {CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void initialize(int* x){unsigned
i=threadIdx.x;if(i<16)x[i]=int(i);}
__global__ void consume(const int* x,int* y){unsigned
i=threadIdx.x;if(i<16)y[i]=x[i]+10;}

int main() {
    try {
        Device<int> x(16),y(16);Stream producer,consumer;Event ready;int
host[16]{};
        initialize<<<1,32,0,producer.s>>>(x.p);CU(cudaGetLastError());

CU(cudaEventRecord(ready.e,producer.s));CU(cudaStreamWaitEvent(consumer.s,rea
dy.e,0));
        consume<<<1,32,0,consumer.s>>>(x.p,y.p);CU(cudaGetLastError());
CU(cudaStreamSynchronize(consumer.s));
CU(cudaMemcpy(host,y.p,sizeof host,cudaMemcpyDeviceToHost));
        for(int i=0;i<16;++i)require(host[i]==i+10,"cross-stream order
failed");
        std::cout << "consumer observed 10 through 25\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

A kernel boundary orders later work in the same stream, but these streams are independent. Recording ready after initialization and making the consumer wait on it transfers the dependency without blocking the host immediately.

The final consumer-stream wait establishes CPU-visible completion. A device-wide wait would be broader than necessary. Destruction occurs after both streams' work is known complete, so no buffer is freed while kernels still reference it.

Expected behavior or output

```
consumer observed10 through25
```

Concrete experiment

Move both kernels to one stream and remove the event; same-stream ordering is sufficient. Do not remove the event while keeping independent streams and then assume a lucky run proves safety.

Verification status: gcc: target required

Technical references: CUDA-ASYNC. See the added references and revision index.

Worked example — Evidence-driven synchronizing parallel gpu work

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
```

optimized path

```
if (expected.size() != observed.size()) return 2;
for (std::size_t i = 0; i < expected.size(); ++i) {
    if (!close_enough(expected[i], observed[i])) return 3;
}
std::cout << "verified " << observed.size() << " values\n";
}
```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only observed with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful

trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?

7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 11 — Overlapping Work with CUDA Streams

This chapter develops **overlapping work with cuda streams** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

11.1 Understanding Asynchronous GPU Operations

11.1.1 Launching Work into Streams

The engineering question in **Launching Work into Streams** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Streams create ordering domains; they do not make unrelated operations safe automatically. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating

memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

11.1.2 Performing Asynchronous Memory Transfers

The engineering question in **Performing Asynchronous Memory Transfers** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Streams create ordering domains; they do not make unrelated operations safe automatically. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Submit an asynchronous copy-kernel-copy chain

Keep page-locked host buffers alive until their stream completes.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
```

```

    T* p=nullptr;
    explicit Pinned(std::size_t n)
    {CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void increment(float* x,unsigned n){unsigned
i=threadIdx.x;if(i<n)x[i]+=1;}

int main() {
    try {
        constexpr unsigned n=64;Pinned<float>
input(n),output(n);Device<float> data(n);Stream stream;
        for(unsigned i=0;i<n;++i)input.p[i]=float(i);

        CU(cudaMemcpyAsync(data.p,input.p,n*sizeof(float),cudaMemcpyHostToDevice,stream.s));
        increment<<<1,64,0,stream.s>>>(data.p,n);CU(cudaGetLastError());

        CU(cudaMemcpyAsync(output.p,data.p,n*sizeof(float),cudaMemcpyDeviceToHost,stream.s));
        unsigned host_work=0;for(unsigned i=0;i<100;++i)host_work+=i;
        CU(cudaStreamSynchronize(stream.s));
        for(unsigned i=0;i<n;++i)require(output.p[i]==float(i+1),"async chain
mismatch");
        std::cout << "last=64 independent-host-sum=" << host_work << '\n';
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

All three operations are queued in one stream, preserving copy-in before kernel before copy-out. Host-side work may run after submission without reading output or overwriting input prematurely.

Pinned allocates page-locked host memory used by the asynchronous transfers. The stream wait—not submission return—is the point after which output inspection and buffer reuse are allowed. This tiny fixture does not establish useful CPU/GPU overlap.

Expected behavior or output

```
last=64 independent-host-sum=4950
```

Concrete experiment

Add timestamps around host work and stream wait separately. A short wait may mean earlier completion, not that the kernel itself took no time.

Verification status: gcc: target required

Technical references: CUDA-ASYNC, CUDA-MEMORY. See the added references and revision index.

11.2 Overlapping Computation and Communication

11.2.1 Using Pinned Host Memory

The engineering question in **Using Pinned Host Memory** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Streams create ordering domains; they do not make unrelated operations safe automatically. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

11.2.2 Building Transfer-Compute Pipelines

The engineering question in **Building Transfer-Compute Pipelines** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Streams create ordering domains; they do not make unrelated operations safe automatically. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Pipeline two chunks with independent buffers and streams

Demonstrate the ownership structure necessary for overlapping one chunk's transfer with another chunk's compute.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p), n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
```

```

};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
{CU(cudaMallocHost(reinterpret_cast<void**>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void scale(float* x,unsigned n){unsigned
i=blockIdx.x*blockDim.x+threadIdx.x;if(i<n)x[i]*=2;}

int main() {
    try {
        constexpr unsigned n=4096;Pinned<float>
host0(n),host1(n);Device<float> d0(n),d1(n);Stream s0,s1;
        for(unsigned i=0;i<n;++i){host0.p[i]=1;host1.p[i]=3;}
        auto submit=[&](float* host,float* device,cudaStream_t stream){

CU(cudaMemcpyAsync(device,host,n*sizeof(float),cudaMemcpyHostToDevice,stream)
);
            scale<<<16,256,0,stream>>>(device,n);CU(cudaGetLastError());

CU(cudaMemcpyAsync(host,device,n*sizeof(float),cudaMemcpyDeviceToHost,stream)
);
            };
            submit(host0.p,d0.p,s0.s);submit(host1.p,d1.p,s1.s);
            CU(cudaStreamSynchronize(s0.s));CU(cudaStreamSynchronize(s1.s));
            for(unsigned i=0;i<n;+
+i)require(host0.p[i]==2&&host1.p[i]==6,"pipeline mismatch");
            std::cout << "chunk0=2 chunk1=6\n";
        } catch(const std::exception& e) {
            std::cerr << e.what() << '\n'; return 1;
        }
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

Every in-flight chunk owns a host buffer, a device buffer, and a stream ordering its three operations. Reusing one device array for both chains would require another dependency and could prevent overlap.

Independent streams permit overlap; they do not guarantee concurrent copy engines or compute. Check device capabilities and inspect a timeline. Both stream

waits precede inspection or destruction, including error-cleanup waits in the stream owners.

Expected behavior or output

```
chunk0=2 chunk1=6
```

Concrete experiment

Add a third chunk only after documenting which completed slot it reuses. Compare one versus two slots at larger n while keeping total transferred data equal.

Verification status: gcc: target required

Technical references: CUDA-ASYNC. See the added references and revision index.

Worked example — Evidence-driven overlapping work with cuda streams

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
```

```

const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
const auto expected = reference_transform(input);
const auto observed = reference_transform(input); // replace with
optimized path

if (expected.size() != observed.size()) return 2;
for (std::size_t i = 0; i < expected.size(); ++i) {
    if (!close_enough(expected[i], observed[i])) return 3;
}
std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only observed with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding

initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?

6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 12 — Handling CUDA Errors and Failures

This chapter develops **handling cuda errors and failures** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

12.1 Detecting Runtime API Errors

12.1.1 Checking CUDA Return Codes

The engineering question in **Checking CUDA Return Codes** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Error handling must distinguish API failures, launch failures, and asynchronous execution failures. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

12.1.2 Building Reusable Error-Checking Utilities

The engineering question in **Building Reusable Error-Checking Utilities** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Error handling must distinguish API failures, launch failures, and asynchronous execution failures. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Attach API operation names to CUDA failures

Query a deliberately out-of-range device ordinal and verify that the checking utility reports the failed API rather than silently continuing.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p), n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
```

```

};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
{CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}
}

```

main.cu

```

#include "cuda_support.hpp"
// No device kernel is needed to test API-return handling.

int main() {
    try {
        int count=0;CU(cudaGetDeviceCount(&count));require(count>0,"a CUDA
device is required");
        bool rejected=false;
        try { CU(cudaSetDevice(count)); }
        catch(const std::runtime_error& e) {
            rejected=std::string(e.what()).find("cudaSetDevice")!
=std::string::npos;
        }
        require(rejected,"invalid ordinal was not diagnosed");
        CU(cudaSetDevice(0));std::cout << "invalid device ordinal rejected
with operation context\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

Valid device ordinals run from 0 through `count-1`. Passing `count` therefore exercises a defined API error rather than corrupting GPU memory. CU preserves the textual call beside `cudaGetErrorString` in the exception.

The surrounding try/catch distinguishes an expected rejection from successful work. Do not discard every exception and declare success: only the particular negative case is caught, while device discovery failures still fail the executable.

Expected behavior or output

```
invalid device ordinal rejected with operation context
```

Concrete experiment

Change the request to count-1 and require success. Keep separate expected-success and expected-failure tests so a broken CUDA installation cannot masquerade as a passed negative case.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

12.2 Detecting Kernel Execution Failures

12.2.1 Checking Launch Errors

The engineering question in **Checking Launch Errors** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Error handling must distinguish API failures, launch failures, and asynchronous execution failures. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

12.2.2 Synchronizing During Debugging

The engineering question in **Synchronizing During Debugging** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Error handling must distinguish API failures, launch failures, and asynchronous execution failures. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating

memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Separate launch configuration errors from execution completion

Trigger a documented-invalid block size, clear that launch error, then run a valid kernel and synchronize explicitly.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" ": "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
```

```

    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
{CU(cudaMallocHost(reinterpret_cast<void**>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void set_one(int* out){if(threadIdx.x==0)out[0]=1;}

int main() {
    try {
        cudaDeviceProp
prop{};CU(cudaGetDeviceProperties(&prop,0));Device<int> result(1);
        set_one<<<1,unsigned(prop.maxThreadsPerBlock+1)>>>(result.p);
        cudaError_t launch=cudaGetLastError();require(launch!=
=cudaSuccess,"oversized block accepted");
        set_one<<<1,1>>>(result.p);CU(cudaGetLastError());
        CU(cudaDeviceSynchronize());int value=0;
        CU(cudaMemcpy(&value,result.p,sizeof value,cudaMemcpyDeviceToHost));
        require(value==1,"valid kernel failed");std::cout << "invalid launch
rejected; valid execution completed\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

cudaGetLastError detects and clears the immediate launch-configuration error. A launch that was accepted can still encounter an execution fault later, which is why the valid path also checks a completion operation.

The fixture avoids deliberate illegal device memory access, which can leave a CUDA context unusable. Diagnose such faults in a disposable process with Compute Sanitizer; do not try to treat a poisoned context as a normal retryable API error.

Expected behavior or output

```
invalid launch rejected; valid execution completed
```

Concrete experiment

Remove only the invalid first launch and run under Compute Sanitizer. Retain both the launch check and completion check; they answer different questions.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

Worked example — Evidence-driven handling cuda errors and failures

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path
```

```

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only `observed` with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result

transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth,

completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?

9. What hardware and software configuration produced the measurement?

10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should

therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 13 — Profiling GPU Applications

This chapter develops **profiling gpu applications** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

13.1 Measuring GPU Execution

13.1.1 Measuring Kernel Duration

The engineering question in **Measuring Kernel Duration** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Profiling is the process of replacing intuition with attributed measurements. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The

corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

13.1.2 Measuring Data-Transfer Costs

The engineering question in **Measuring Data-Transfer Costs** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Profiling is the process of replacing intuition with attributed measurements. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Time transfer and kernel intervals with stream events

Measure separate CUDA-event intervals while validating the final output; durations depend on hardware and are not prefilled.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
```

```

{CU(cudaMallocHost(reinterpret_cast<void**>(&p),n*sizeof(T)));}
~Pinned(){(void)cudaFreeHost(p);}
Pinned(const Pinned&)=delete;
Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void square(float* x,unsigned n){unsigned
i=blockIdx.x*blockDim.x+threadIdx.x;if(i<n)x[i]*=x[i];}

int main() {
    try {
        constexpr unsigned n=1u<<20;Pinned<float> host(n);Device<float>
data(n);Stream stream;Event start,copied,computed;
        for(unsigned i=0;i<n;++i)host.p[i]=2;
        CU(cudaFree(nullptr)); // initialize runtime before the measured
chain
        CU(cudaEventRecord(start.e,stream.s));

        CU(cudaMemcpyAsync(data.p,host.p,n*sizeof(float),cudaMemcpyHostToDevice,stream.s));
        CU(cudaEventRecord(copied.e,stream.s));
        square<<<n/256,256,0,stream.s>>>(data.p,n);CU(cudaGetLastError());

        CU(cudaEventRecord(computed.e,stream.s));CU(cudaEventSynchronize(computed.e))
        ;
        float
copy_ms=0,kernel_ms=0;CU(cudaEventElapsedTime(&copy_ms,start.e,copied.e));
        CU(cudaEventElapsedTime(&kernel_ms,copied.e,computed.e));
        CU(cudaMemcpy(host.p,data.p,n*sizeof(float),cudaMemcpyDeviceToHost));
        for(unsigned i=0;i<n;++i)require(host.p[i]==4,"timed square failed");
        std::cout << "copy_ms=" << copy_ms << " kernel_ms=" << kernel_ms << "
verified=1048576\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

Three events bracket two different operations in one ordering domain. Waiting for `computed` ensures both measured intervals have finished before calling `cudaEventElapsedTime`. Allocations and host initialization are outside these intervals.

The final readback is deliberately not included in either reported number. Thus neither number is end-to-end request latency. Repeat warm-up and measurement cycles for a benchmark; the single diagnostic measurement may include first-kernel loading costs.

Expected behavior or output

```
copy_ms=<measured milliseconds> kernel_ms=<measured milliseconds>  
verified=1048576
```

Concrete experiment

Add a fourth event after an asynchronous device-to-host copy, then report that interval separately and compare their sum with a host steady-clock interval.

Verification status: gcc: target required

Technical references: CUDA-ASYNC. See the added references and revision index.

13.2 Locating GPU Bottlenecks

13.2.1 Identifying Memory-Bound Kernels

The engineering question in **Identifying Memory-Bound Kernels** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Profiling is the process of replacing intuition with attributed measurements. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements

should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

13.2.2 Identifying Compute-Bound Kernels

The engineering question in **Identifying Compute-Bound Kernels** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Profiling is the process of replacing intuition with attributed measurements. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Calculate arithmetic intensity for two explicit kernels

Use a roofline-style calculation to distinguish a streaming SAXPY from a reused-data computation; rates here are illustrative inputs.

main.cpp

```
#include <iomanip>
#include <iostream>
#include <algorithm>

int main() {
    struct Work {const char* name;double flops,bytes;};
    const Work workloads[]{{"saxpy",2,12},{ "reused-tile",512,16}};
    constexpr double bandwidth_gbs=100, compute_gflops=2000;
    for(auto w:workloads){
        double intensity=w.flops/w.bytes;
        double bound=std::min(compute_gflops,bandwidth_gbs*intensity);
        std::cout << w.name << " intensity=" << std::fixed <<
std::setprecision(3)
                << intensity << " bound_GFLOP/s=" << bound << '\n';
    }
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

SAXPY reads x and writes y : with float32 data that is 12 modeled bytes for one multiply plus one add. Multiplying arithmetic intensity by memory bandwidth produces the bandwidth-side throughput bound.

The hypothetical reused tile performs 512 operations per 16 external bytes, so the 2000 GFLOP/s compute ceiling wins. These are analytic ceilings under assumed

traffic/rates, not GPU measurements; use a profiler to establish actual bytes, instructions, and bottlenecks.

Expected behavior or output

```
saxpy intensity=0.167 bound_GFLOP/s=16.667
reused-tile intensity=32.000 bound_GFLOP/s=2000.000
```

Concrete experiment

Double assumed bandwidth without changing compute peak. SAXPY's bound doubles while the reused-tile bound stays 2000; explain why increasing occupancy alone does not change this calculation.

Verification status: gcc: passed

Technical references: CPP20, NSIGHT-COMPUTE. See the added references and revision index.

Worked example — Evidence-driven profiling gpu applications

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}
```

```

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only observed with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding

initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?

6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 14 — Optimizing CUDA Kernel Performance

This chapter develops **optimizing cuda kernel performance** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

14.1 Improving Occupancy

14.1.1 Understanding Register Pressure

The engineering question in **Understanding Register Pressure** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Optimization should follow measured limiting resources rather than generic tuning folklore. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

14.1.2 Understanding Shared-Memory Limits

The engineering question in **Understanding Shared-Memory Limits** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Optimization should follow measured limiting resources rather than generic tuning folklore. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Query occupancy for alternative dynamic shared-memory sizes

Ask the runtime how many blocks of a particular kernel can reside per multiprocessor for two shared-memory budgets.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p), n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
```

```

};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
{CU(cudaMallocHost(reinterpret_cast<void**>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void scratch_kernel(float* out){
    extern __shared__ float scratch[];
    unsigned t=threadIdx.x;scratch[t]=float(t);__syncthreads();
    if(t==0)out[blockIdx.x]=scratch[blockDim.x-1];
}

int main() {
    try {
        cudaDeviceProp prop{};CU(cudaGetDeviceProperties(&prop,0));
        int small=0,large=0;constexpr int threads=128;
        std::size_t
        larger=std::min<std::size_t>(16384,prop.sharedMemPerBlock);

        CU(cudaOccupancyMaxActiveBlocksPerMultiprocessor(&small,scratch_kernel,thread
s,threads*sizeof(float)));

        CU(cudaOccupancyMaxActiveBlocksPerMultiprocessor(&large,scratch_kernel,thread
s,larger));

        require(small>0 && large>0,"kernel has no resident blocks");
        std::cout << "resident-blocks small=" << small << " large=" << large
<< '\n';
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 -Xptxas=-v main.cu -o example
./example

```

Explanation

The query is kernel-specific: it considers compiled register/resource use as well as block size and dynamic shared bytes. Increasing the shared-memory reservation can reduce residency even when the kernel logically uses only 128 floats.

Resident blocks are not achieved throughput. Register pressure, scheduling limits, and memory behavior can dominate, and occupancy is not a goal to maximize at all costs. Record the actual device and compile resource report beside the query.

Expected behavior or output

```
resident-blocks small=<device-dependent count> large=<device-dependent count>
```

Concrete experiment

Change threads to 256 and adjust the minimum shared allocation. Compare occupancy predictions with measured runtime for real work before choosing a launch size.

Verification status: gcc: target required

Technical references: CUDA-OCCUPANCY, CUDA-GUIDE. See the added references and revision index.

14.2 Optimizing Instruction Throughput

14.2.1 Reducing Expensive Operations

The engineering question in **Reducing Expensive Operations** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Optimization should follow measured limiting resources rather than generic tuning folklore. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements

should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

14.2.2 Improving Arithmetic Intensity

The engineering question in **Improving Arithmetic Intensity** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Optimization should follow measured limiting resources rather than generic tuning folklore. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Hoist one reciprocal while checking the numerical change

Compare elementwise division with multiplication by a host-computed reciprocal for a common nonzero divisor.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
```

```

};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
    {CU(cudaMallocHost(reinterpret_cast<void**>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void divide(const float* x,float* y,float divisor,unsigned n)
{unsigned i=threadIdx.x;if(i<n)y[i]=x[i]/divisor;}
__global__ void multiply(const float* x,float* y,float reciprocal,unsigned n)
{unsigned i=threadIdx.x;if(i<n)y[i]=x[i]*reciprocal;}

int main() {
    try {
        float input[32],a[32] {},b[32] {};for(unsigned i=0;i<32;+
+i)input[i]=float(i)-7;
        Device<float> x(32),da(32),db(32);CU(cudaMemcpy(x.p,input,sizeof
input,cudaMemcpyHostToDevice));
        divide<<<1,32>>>(x.p,da.p,3.0f,32);CU(cudaGetLastError());
        multiply<<<1,32>>>(x.p,db.p,1.0f/3.0f,32);CU(cudaGetLastError());
        CU(cudaMemcpy(a,da.p,sizeof
a,cudaMemcpyDeviceToHost));CU(cudaMemcpy(b,db.p,sizeof
b,cudaMemcpyDeviceToHost));
        float error=0;for(unsigned i=0;i<32;+
+i)error=std::max(error,std::abs(a[i]-b[i]));
        require(error<=1e-6f,"reciprocal approximation exceeds fixture
budget");
        std::cout << "maximum error <= 1e-6 for32 values\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

The divisor is common to all elements, so a reciprocal can be computed once instead of requested as a division per lane. The two expressions may round differently; the absolute-error check is part of the transformation's acceptance criteria.

The compiler may already replace or optimize divisions depending on flags. Inspect generated instructions and benchmark a substantial workload before claiming improvement. Zero, tiny divisors, infinities, and NaNs require a separate input contract; this fixture uses the explicit finite divisor3.

Expected behavior or output

```
maximum error <= 1e-6 for32 values
```

Concrete experiment

Use divisors2 and10. Predict exact agreement for the power-of-two scaling when representable, then measure error for10. Do not change fast-math flags without revalidating numerical behavior.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

Worked example — Evidence-driven optimizing cuda kernel performance

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
```

```

        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only `observed` with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 15 — Dividing Applications Between CPU and GPU

This chapter develops **dividing applications between cpu and gpu** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

15.1 Identifying GPU-Appropriate Workloads

15.1.1 Recognizing Highly Parallel Operations

The engineering question in **Recognizing Highly Parallel Operations** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Heterogeneous design is a partitioning problem across control, data movement, and throughput work. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

15.1.2 Avoiding GPU Offload for Tiny Workloads

The engineering question in **Avoiding GPU Offload for Tiny Workloads** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Heterogeneous design is a partitioning problem across control, data movement, and throughput work. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Find an offload break-even point from measured-model inputs

Compute a decision table from assumed setup and per-element times rather than assuming the GPU always wins.

main.cpp

```
#include <array>
#include <iostream>

int main() {
    constexpr double cpu_ns=10,gpu_ns=1,setup_ns=50000;
    for(int n:std::array{100,1000,10000}){
        double cpu=n*cpu_ns,gpu=setup_ns+n*gpu_ns;
        std::cout << n << " elements -> " << (gpu<cpu?"gpu":"cpu") << '\n';
    }
    std::cout << "break-even above=" << setup_ns/(cpu_ns-gpu_ns) << "
elements\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The model charges a fixed 50000ns setup/transfer cost to GPU work and a lower per-element cost. Solving $\text{setup} + n * \text{gpu} < n * \text{cpu}$ produces a break-even count near 5555.56.

These constants are invented inputs for the decision exercise. In a real pipeline measure them for the actual shape and transfer policy; resident GPU data or multiple fused operations can shift the break-even point substantially.

Expected behavior or output

```
100 elements -> cpu
1000 elements -> cpu
```

```
10000 elements -> gpu  
break-even above=5555.56 elements
```

Concrete experiment

Set `setup_ns` to 5000 to model amortized setup. The 1000-element case should now choose GPU. Explain why reusing device-resident data changes the decision.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

15.2 Designing Heterogeneous Pipelines

15.2.1 Assigning Control Logic to the CPU

The engineering question in **Assigning Control Logic to the CPU** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Heterogeneous design is a partitioning problem across control, data movement, and throughput work. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating

memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

15.2.2 Assigning Throughput Work to the GPU

The engineering question in **Assigning Throughput Work to the GPU** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Heterogeneous design is a partitioning problem across control, data movement, and throughput work. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Keep batch validation and aggregation outside a compute backend

Separate CPU control decisions from the elementwise operation that a GPU adapter could implement.

main.cpp

```
#include <span>
#include <vector>
#include <stdexcept>
#include <iostream>
#include <numeric>

int main() {
    auto backend=[](std::span<const float> in,std::span<float> out){
        if(in.size()!=out.size())throw std::invalid_argument("shape");
        for(std::size_t i=0;i<in.size();++i)out[i]=in[i]*in[i];
    };
    std::vector<float> batch{1,2,3,4},output(batch.size());
    if(batch.empty()||batch.size()>1024)throw std::invalid_argument("batch
limit");
    backend(batch,output);
    std::cout << "accepted=" << batch.size() << " energy="
        << std::accumulate(output.begin(),output.end(),0.0f) << '\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Admission is a CPU-side policy: zero-sized or oversized batches are rejected before dispatch. The backend contract specifies equal-sized input/output spans and a synchronous completion boundary.

The elementwise squares are the throughput operation. Aggregating the completed output is an application decision; a real asynchronous GPU adapter would need a future/event boundary before that aggregation. This test validates the interface with a CPU implementation, not GPU execution.

Expected behavior or output

```
accepted=4 energy=30
```

Concrete experiment

Add an asynchronous adapter returning a future and require callers to await it before reading output. Keep size validation before submitting any work.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Evidence-driven dividing applications between cpu and gpu

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path
}
```



```

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only `observed` with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result

transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth,

completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?

9. What hardware and software configuration produced the measurement?

10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should

therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 16 — Scaling Across Multiple GPUs

This chapter develops **scaling across multiple gpus** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

16.1 Discovering and Selecting GPU Devices

16.1.1 Querying Device Capabilities

The engineering question in **Querying Device Capabilities** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Multiple GPUs introduce placement, communication, and result-combination contracts. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The

corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

16.1.2 Assigning Work to Specific Devices

The engineering question in **Assigning Work to Specific Devices** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Multiple GPUs introduce placement, communication, and result-combination contracts. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Enumerate devices and verify allocation on the selected ordinal

Report runtime-discovered capabilities and exercise one explicit device selection.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" ": +cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
    {CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
```



```

~Pinned(){(void)cudaFreeHost(p);}
Pinned(const Pinned&)=delete;
Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
// Device discovery is a host runtime operation.

int main() {
    try {
        int count=0;CU(cudaGetDeviceCount(&count));require(count>0,"no
devices");
        for(int i=0;i<count;++i){cudaDeviceProp
p{};CU(cudaGetDeviceProperties(&p,i));
            std::cout << "device=" << i << " name=" << p.name << " compute="
<< p.major << '.' << p.minor
                << " global-bytes=" << p.totalGlobalMem << '\n';}
            const int selected=count-1;CU(cudaSetDevice(selected));
            {Device<int>
data(1);CU(cudaMemset(data.p,0,sizeof(int)));CU(cudaDeviceSynchronize());}
            int
current=-1;CU(cudaGetDevice(&current));require(current==selected,"wrong
active device");
            std::cout << "selected=" << current << '\n';
        } catch(const std::exception& e) {
            std::cerr << e.what() << '\n'; return 1;
        }
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

Device indices are discovered, not hardcoded from a workstation inventory. Compute capability and memory capacity describe support/resources but are not an application performance score.

`cudaSetDevice` affects the calling host thread's active CUDA device. Allocation and its destruction occur while that device is selected. With multiple host threads, establish each thread's device context rather than assuming one call globally selects it for everyone.

Expected behavior or output

```
One capability line per visible device, followed by selected=<last ordinal>.
Names and capacities depend on the host.
```

Concrete experiment

Accept an ordinal argument and reject values outside $0 \dots \text{count}-1$ before allocation. Test the rejection on a one-GPU machine as well as device selection on a multi-GPU machine.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

16.2 Distributing Work Across GPUs

16.2.1 Partitioning Independent Data

The engineering question in **Partitioning Independent Data** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Multiple GPUs introduce placement, communication, and result-combination contracts. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

16.2.2 Combining Results from Multiple Devices

The engineering question in **Combining Results from Multiple Devices** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Multiple GPUs introduce placement, communication, and result-combination contracts. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating

memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Partition an array across two devices with explicit ownership

Use independent device allocations for two five-element shards, then combine host results.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run. Requires at least two NVIDIA GPUs; a one-GPU host is a skipped prerequisite, not a passed multi-GPU test.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+": "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
```

```

        cudaEvent_t e{};
        Event(){CU(cudaEventCreate(&e));}
        ~Event(){(void)cudaEventDestroy(e);}
        Event(const Event&)=delete;
        Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
{CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}

```

main.cu

```

#include "cuda_support.hpp"
__global__ void add_offset(int* out,int begin,int count){int
i=int(threadIdx.x);if(i<count)out[i]=(begin+i)*2;}

int main() {
    try {
        int
        devices=0;CU(cudaGetDeviceCount(&devices));require(devices>=2,"two visible
        GPUs are required");
        int combined[10]{};
        for(int ordinal=0;ordinal<2;++ordinal){
            CU(cudaSetDevice(ordinal));
            Device<int> shard(5);
            add_offset<<<1,32>>>(shard.p,ordinal*5,5);CU(cudaGetLastError());

            CU(cudaMemcpy(combined+ordinal*5,shard.p,5*sizeof(int),cudaMemcpyDeviceToHost
            ));
        }
        for(int i=0;i<10;++i)require(combined[i]==2*i,"shard placement
        mismatch");
        std::cout << "two-device result: 0 2 4 6 8 10 12 14 16 18\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

The host assigns disjoint logical ranges through `begin`. Each allocation is created, used, copied, and destroyed on its selected device. Host combination writes the corresponding disjoint result range.

The loop intentionally executes shards sequentially to make device ownership clear. It demonstrates multi-device placement, not multi-device speedup. True overlap requires separate host submission threads or careful asynchronous ownership, and may be limited by shared transfer bandwidth.

Expected behavior or output

```
two-device result: 0 2 4 6 8 10 12 14 16 18
```

Concrete experiment

Use two host threads, one per device, with separate errors captured and joined before validation. Compare wall time only for substantially larger arrays.

Verification status: gcc: target required

Technical references: [CUDA-GUIDE](#), [CUDA-MEMORY](#). See the added references and revision index.

Worked example — Evidence-driven scaling across multiple gpus

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}
```

```

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only observed with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding

initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?

6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 17 — Processing Signals with Modern C++

This chapter develops **processing signals with modern c++** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

17.1 Representing Sampled Signals

17.1.1 Representing Real-Valued Samples

The engineering question in **Representing Real-Valued Samples** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Signal processing requires explicit sample rate, units, complex conventions, and boundary handling. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

17.1.2 Representing Complex I/Q Samples

The engineering question in **Representing Complex I/Q Samples** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Signal processing requires explicit sample rate, units, complex conventions, and boundary handling. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Represent a quadrature tone as complex samples

Construct a four-sample complex sinusoid and check magnitude and average power using ordinary C++ complex arithmetic.

main.cpp

```
#include <array>
#include <complex>
#include <numbers>
#include <cmath>
#include <iostream>

int main() {
    std::array<std::complex<double>,4> iq{};double power=0;
    for(std::size_t n=0;n<iq.size();++n){
        double phase=2*std::numbers::pi*double(n)/4;
        iq[n]=std::polar(2.0,phase);power+=std::norm(iq[n]);
        if(std::abs(std::abs(iq[n])-2)>1e-12)return 1;
    }
    std::cout << "samples=" << iq.size() << " amplitude=2 mean-power=" <<
    power/iq.size() << '\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

`std::complex<double>` stores real I and imaginary Q components of a general sampled signal. `polar(2,phase)` specifies amplitude2 and phase in radians; one full revolution spans these four sample positions.

`abs` returns magnitude, whereas `norm` returns squared magnitude. Thus every sample has magnitude2 and power4. This is a discrete mathematical tone; physical voltage/power units require a separately specified calibration and impedance.

Expected behavior or output

```
samples=4 amplitude=2 mean-power=4
```

Concrete experiment

Set amplitude to 0.5. Predict mean-power 0.25 and keep the magnitude assertion consistent. Changing sample rate alone changes physical frequency interpretation, not these four stored values.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

17.2 Implementing Fundamental Signal Operations

17.2.1 Applying Filters

The engineering question in **Applying Filters** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Signal processing requires explicit sample rate, units, complex conventions, and boundary handling. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

17.2.2 Calculating Correlations and Convolutions

The engineering question in **Calculating Correlations and Convolutions** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Signal processing requires explicit sample rate, units, complex conventions, and boundary handling. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating

memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Distinguish causal convolution from lagged correlation

Compute a full FIR convolution and nonnegative-lag correlation on a three-sample signal using explicitly different index formulas.

main.cpp

```
#include <array>
#include <iostream>

int main() {
    const std::array<int,3>x{1,2,3};const std::array<int,2>h{1,-1};
    std::array<int,4>convolution{};
    for(std::size_t n=0;n<convolution.size();++n)
        for(std::size_t k=0;k<h.size();++k)
            if(n>=k && n-k<x.size())convolution[n]+=h[k]*x[n-k];
    std::array<int,3>correlation{};
    for(std::size_t lag=0;lag<3;++lag)
        for(std::size_t i=0;i+lag<3;++i)correlation[lag]+=x[i]*x[i+lag];
    std::cout << "convolution:";for(int v:convolution)std::cout << ' ' << v;
    std::cout << "\ncorrelation:";for(int v:correlation)std::cout << ' ' <<
v;std::cout << '\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The convolution evaluates $h[k] * x[n-k]$ and treats outside-signal indices as zero. Its length is $3+2-1=4$, so the trailing -3 is part of the full response rather than silently discarded.

Correlation uses a positive lag to compare $x[i]$ with $x[i+lag]$. For complex signals one factor must be conjugated according to the chosen convention. These real integer fixtures keep index direction and boundary handling inspectable.

Expected behavior or output

```
convolution: 1 1 1 -3
correlation: 14 8 3
```

Concrete experiment

Replace `h` with `{1,1}`. Predict `1,3,5,3`. Do not change the correlation formula merely because the filter changed; these are different operations.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Evidence-driven processing signals with modern c++

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized_path
```

```

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only `observed` with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result

transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth,

completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?

9. What hardware and software configuration produced the measurement?

10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should

therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 18 — Computing Frequency-Domain Representations

This chapter develops **computing frequency-domain representations** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

18.1 Understanding the Discrete Fourier Transform

18.1.1 Converting Time-Domain Samples to Frequency Components

The engineering question in **Converting Time-Domain Samples to Frequency Components** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Frequency-domain processing adds scaling, bin ordering, windowing, and numerical interpretation. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

18.1.2 Understanding Magnitude and Phase

The engineering question in **Understanding Magnitude and Phase** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Frequency-domain processing adds scaling, bin ordering, windowing, and numerical interpretation. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Locate the nonzero bin of a complex tone with a direct DFT

Compute the forward unnormalized DFT of eight samples with a positive-frequency complex tone at bin2.

main.cpp

```
#include <array>
#include <complex>
#include <numbers>
#include <cmath>
#include <iostream>

int main() {
    constexpr std::size_t
    n=8;std::array<std::complex<double>,n>x{},spectrum{};
    for(std::size_t t=0;t<n;+
+t)x[t]=std::polar(1.0,2*std::numbers::pi*2*double(t)/n);
    for(std::size_t k=0;k<n;++k)for(std::size_t t=0;t<n;++t)
        spectrum[k]+=x[t]*std::polar(1.0,-2*std::numbers::pi*double(k*t)/n);
    for(std::size_t k=0;k<n;++k)
        if(std::abs(spectrum[k])-(k==2?
std::complex<double>{8,0}:std::complex<double>{0,0}))>1e-12)return 1;
    std::cout << "bin2 magnitude=8 phase approximately0; other bins
approximately0\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The signal phase uses a positive sign and the forward transform uses a negative sign. At $k=2$ the phases cancel, so eight ones sum to 8. Other bins cancel within floating-point tolerance.

No $1/N$ normalization is applied. Bin2 corresponds to $2 \cdot \text{sample_rate}/8$ in physical frequency. Magnitude and phase describe a complex coefficient; phase of near-zero bins is not meaningful and should not be interpreted.

Expected behavior or output

```
bin2 magnitude=8 phase approximately0; other bins approximately0
```

Concrete experiment

Change the signal bin to 3 and update the oracle. Then negate the signal phase: its peak should move to bin $N-3$ rather than staying at 3.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

18.2 Applying Fast Fourier Transforms

18.2.1 Using CPU FFT Libraries

The engineering question in **Using CPU FFT Libraries** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Frequency-domain processing adds scaling, bin ordering, windowing, and numerical interpretation. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

18.2.2 Using GPU FFT Libraries

The engineering question in **Using GPU FFT Libraries** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Frequency-domain processing adds scaling, bin ordering, windowing, and numerical interpretation. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Check FFTW and cuFFT against the same impulse transform

Two complete library calls transform{1,0,0,0}. Every complex output must be $1+0i$, giving an independent analytic oracle instead of comparing two potentially mismatched conventions.

FFTW development headers and CUDA/cuFFT were not available here. Both programs are unexecuted target integrations, not placeholder API names.

fftw_example.cpp

```
#include <fftw3.h>
#include <cmath>
#include <iostream>
int main(){
    auto* x=fftw_alloc_complex(4);auto* y=fftw_alloc_complex(4);
    if(!x||!y){fftw_free(x);fftw_free(y);return 1;}
    for(int i=0;i<4;++i){x[i][0]=(i==0);x[i][1]=0;}
    auto plan=fftw_plan_dft_1d(4,x,y,FFTW_FORWARD,FFTW_ESTIMATE);
    if(!plan){fftw_free(x);fftw_free(y);return 2;}
    fftw_execute(plan);bool valid=true;
    for(int i=0;i<4;++i)valid &= std::abs(y[i][0]-1)<1e-12&&std::abs(y[i]
[1])<1e-12;
    fftw_destroy_plan(plan);fftw_free(x);fftw_free(y);
    if(!valid)return 3;std::cout<<"FFTW impulse bins=1+0i\n";
}
```

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e,const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
```

```

    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
    {CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}
}

```

cufft_example.cu

```

#include "cuda_support.hpp"
#include <cufft.h>
void fftcheck(cufftResult r){if(r!=CUFFT_SUCCESS)throw
std::runtime_error("cuFFT error");}
struct Plan{cufftHandle p{};Plan(){fftcheck(cufftPlan1d(&p,4,CUFFT_C2C,1));}
~Plan(){(void)cufftDestroy(p);}};
int main(){try{
    cufftComplex input[4]{{1,0},{0,0},{0,0},{0,0}},output[4]{};
    Device<cufftComplex>x(4),y(4);Plan plan;
    CU(cudaMemcpy(x.p,input,sizeof input,cudaMemcpyHostToDevice));
    fftcheck(cufftExecC2C(plan.p,x.p,y.p,CUFFT_FORWARD));
    CU(cudaMemcpy(output,y.p,sizeof output,cudaMemcpyDeviceToHost));
    for(auto v:output)require(std::abs(v.x-1)<1e-6f&&std::abs(v.y)<1e-6f,"FFT
mismatch");
    std::cout<<"cuFFT impulse bins=1+0i\n";
}catch(const std::exception&e){std::cerr<<e.what()<<"\n";return 1;}}

```

Run or inspect

```

g++ -std=c++20 fftw_example.cpp -lfftw3 -o fftw_example
./fftw_example

```

```
nvcc -std=c++17 cufft_example.cu -lcufft -o cufft_example
./cufft_example
```

Explanation

FFTW separates plan creation from execution and uses library-aligned complex storage. FFTW_ESTIMATE avoids measurement-based planning that may overwrite input during plan creation. Destruction releases the plan and both arrays.

cuFFT's C2C plan operates on device complex-float buffers; FFTW's example uses complex doubles. Both forward transforms are unnormalized and use an impulse oracle, but their tolerances differ with precision. Plan creation, execution, and transfer must be timed separately when comparing performance.

Expected behavior or output

```
FFTW impulse bins=1+0i
cuFFT impulse bins=1+0i
```

Concrete experiment

Use a constant sequence{1,1,1,1}. Expect DC=4+0i and all other bins approximately zero. Then add inverse transforms and explicitly divide by 4 to recover the input.

Verification status: gcc: target required

Technical references: FFTW, CUFFT. See the added references and revision index.

Worked example — Evidence-driven computing frequency-domain representations

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
```

```

    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only `observed` with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 19 — Designing Real-Time Processing Systems

This chapter develops **designing real-time processing systems** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

19.1 Understanding Real-Time Requirements

19.1.1 Distinguishing Hard and Soft Real-Time Behavior

The engineering question in **Distinguishing Hard and Soft Real-Time Behavior** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Real-time engineering is about bounded timing behavior, not simply high average speed. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

19.1.2 Defining Latency and Deadline Requirements

The engineering question in **Defining Latency and Deadline Requirements** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Real-time engineering is about bounded timing behavior, not simply high average speed. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Separate a deadline miss from a high average rate

Classify an explicit measured-duration fixture against a 10ms relative deadline.

main.cpp

```
#include <array>
#include <chrono>
#include <iostream>

int main() {
    using namespace std::chrono_literals;
    const std::array durations{8ms,9ms,14ms,7ms,8ms};
    constexpr auto deadline=10ms;unsigned misses=0;
    std::chrono::milliseconds total{};
    for(auto d:durations){total+=d;if(d>deadline)++misses;}
    std::cout << "mean_ms=" << double(total.count())/durations.size()
               << " misses=" << misses << '/' << durations.size() << '\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The 8,9,14,7,8ms values are an illustrative timing trace. Their average is 9.2ms, which is below the deadline, but the 14ms operation still misses its own requirement. Average throughput cannot establish deadline compliance.

A hard real-time requirement needs a justified bound under the supported system model; a soft requirement may permit a specified miss rate and recovery policy. This finite trace supports neither universal claim—it only demonstrates how to count violations.

Expected behavior or output

```
mean_ms=9.2 misses=1/5
```

Concrete experiment

Use a 20ms deadline and observe zero misses. Explain why changing the requirement is not the same as optimizing the 14ms execution.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

19.2 Reducing Timing Variability

19.2.1 Avoiding Unpredictable Allocation

The engineering question in **Avoiding Unpredictable Allocation** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Real-time engineering is about bounded timing behavior, not simply high average speed. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

19.2.2 Controlling Blocking and Scheduling

The engineering question in **Controlling Blocking and Scheduling** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Real-time engineering is about bounded timing behavior, not simply high average speed. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Preallocate a filter state and process without hot-path allocation

Use fixed arrays for a four-sample moving sum; configuration and processing have separate phases.

main.cpp

```
#include <array>
#include <cstdint>
#include <iostream>

int main() {
    struct MovingSum{std::array<int,4> delay{};std::size_t next=0;int sum=0;
        int process(int x){sum-
=delay[next];delay[next]=x;sum+=x;next=(next+1)%delay.size();return sum;}};
    MovingSum filter;
    const std::array<int,6> input{1,2,3,4,5,6};
    std::array<int,6> output{};
    for(std::size_t i=0;i<input.size();+
+i)output[i]=filter.process(input[i]);
    std::cout << "moving sums:";for(int x:output)std::cout << ' ' <<
x;std::cout << '\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

delay and output storage are allocated before processing. Each call performs a constant number of operations, replaces one ring slot, and updates the sum without a heap allocation or blocking primitive.

Removing allocation is one source of predictability, not proof of a wall-time bound. Cache state, scheduling, interrupts, and platform frequency still matter. The integer range is small enough for this fixture; production accumulators require range analysis.

Expected behavior or output

```
moving sums: 1 3 6 10 14 18
```

Concrete experiment

Change the window to three and predict 1,3,6,9,12,15. Keep the wrap modulus tied to `delay.size()` so the storage and algorithm cannot disagree.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Evidence-driven designing real-time processing systems

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
}
```

```
std::cout << "verified " << observed.size() << " values\n";  
}
```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only `observed` with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A

number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?

10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 20 — Building High-Throughput Data Pipelines

This chapter develops **building high-throughput data pipelines** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

20.1 Designing Streaming Processing Stages

20.1.1 Separating Acquisition, Processing, and Output

The engineering question in **Separating Acquisition, Processing, and Output** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Streaming systems require explicit queue capacity and overload behavior. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

20.1.2 Buffering Data Between Stages

The engineering question in **Buffering Data Between Stages** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Streaming systems require explicit queue capacity and overload behavior. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Carry sequence and sampling time through three explicit stages

Model acquisition, processing, and output as separate functions exchanging owned frames.

main.cpp

```
#include <array>
#include <cstdint>
#include <iostream>

int main() {
    struct Frame{std::uint64_t sequence;std::uint64_t
start_ns;std::array<float,4> samples;};
    auto acquire=[](){return Frame{7,1000000,{1,2,3,4}};};
    auto process=[](Frame f){for(auto&v:f.samples)v*=0.5f;return f;};
    auto output=[](const Frame&f){float sum=0;for(float v:f.samples)sum+=v;
        std::cout<<"sequence="<<f.sequence<<" sample-time="<<f.start_ns<<"
sum="<<sum<<"\n";};
    output(process(acquire()));
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Frame owns its four samples and carries acquisition metadata. Passing by value makes the stage ownership transfer explicit for this small model; processing modifies values but preserves the source sequence and timestamp.

The sequential calls are not an asynchronous pipeline. Adding queues later must retain these ownership and metadata rules rather than replacing acquisition time with queue-arrival time. Output computes a summary only after processing has completed.

Expected behavior or output

```
sequence=7 sample-time=1000000 sum=5
```

Concrete experiment

Make acquisition produce two frames with consecutive sequence IDs and 4ms-spaced timestamps. Insert a bounded queue between stages and verify neither timestamp is rewritten.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

20.2 Maintaining Pipeline Throughput

20.2.1 Applying Backpressure

The engineering question in **Applying Backpressure** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Streaming systems require explicit queue capacity and overload behavior. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

20.2.2 Measuring Stage-Level Bottlenecks

The engineering question in **Measuring Stage-Level Bottlenecks** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. Streaming systems require explicit queue capacity and overload behavior. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The

corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Apply a bounded admission policy and identify the slowest stage

Simulate five arrivals into a capacity-two queue while a consumer removes one item after every second arrival.

main.cpp

```
#include <deque>
#include <array>
#include <iostream>
#include <algorithm>

int main() {
    std::deque<int> queue; unsigned rejected=0, high_water=0, completed=0;
    for(int id=0; id<5; ++id){
        if(queue.size()==2) ++rejected; else queue.push_back(id);
        high_water=std::max(high_water, unsigned(queue.size()));
        if(id%2==1 && !queue.empty()) {queue.pop_front(); ++completed;}
    }
    const std::array<int, 3> stage_us{100, 400, 150};
    std::cout<<"completed="<<completed<<" queued="<<queue.size()<<"
    rejected="<<rejected
              <<" high-water="<<high_water<<"
    bottleneck_us="<<*std::max_element(stage_us.begin(), stage_us.end())<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The third arrival fills the slot freed after arrival1; arrival3 is rejected before the second dequeue. The queue cannot grow past two, so excess load becomes an explicit rejection instead of unbounded latency.

The illustrative 400µs processing stage sets a 2500-items/s serial-stage capacity bound, assuming independent balanced staging and no other costs. Enlarging the

queue changes buffering, not that service rate. Real measurements must include queue residence and overlap.

Expected behavior or output

```
completed=2 queued=2 rejected=1 high-water=2 bottleneck_us=400
```

Concrete experiment

Consume after every arrival instead. Rejections should disappear for this trace. Then distinguish changing consumption capacity from merely raising the queue limit.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Evidence-driven building high-throughput data pipelines

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
```

```

const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
const auto expected = reference_transform(input);
const auto observed = reference_transform(input); // replace with
optimized path

if (expected.size() != observed.size()) return 2;
for (std::size_t i = 0; i < expected.size(); ++i) {
    if (!close_enough(expected[i], observed[i])) return 3;
}
std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only observed with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding

initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?

6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 21 — Accelerating Signal Processing on GPUs

This chapter develops **accelerating signal processing on gpus** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

21.1 Mapping DSP Operations to CUDA

21.1.1 Parallelizing Vector and Matrix Operations

The engineering question in **Parallelizing Vector and Matrix Operations** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. GPU DSP pipelines must include acquisition, transfer, execution, and result delivery in latency accounting. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

21.1.2 Parallelizing FFT-Based Processing

The engineering question in **Parallelizing FFT-Based Processing** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. GPU DSP pipelines must include acquisition, transfer, execution, and result delivery in latency accounting. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Execute a two-by-two complex linear transform on the GPU

Map output rows of a small complex matrix-vector transform to independent threads and validate real/imaginary components.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p), n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
```

```

};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
{CU(cudaMallocHost(reinterpret_cast<void*>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}
}

```

main.cu

```

#include "cuda_support.hpp"
struct Complex{float re,im;};
__device__ Complex multiply(Complex a,Complex b){return{a.re*b.re-
a.im*b.im,a.re*b.im+a.im*b.re};}
__global__ void transform(const Complex* matrix,const Complex* x,Complex* y){
    unsigned row=threadIdx.x;if(row>=2)return;Complex sum{0,0};
    for(unsigned k=0;k<2;++k){auto
v=multiply(matrix[row*2+k],x[k]);sum.re+=v.re;sum.im+=v.im;}y[row]=sum;
}

int main() {
    try {
        Complex matrix[4]{{1,0},{1,0},{1,0},{-1,0}},input[2]{{2,1},
{1,-1}},output[2]{};
        Device<Complex>a(4),x(2),y(2);
        CU(cudaMemcpy(a.p,matrix,sizeof
matrix,cudaMemcpyHostToDevice));CU(cudaMemcpy(x.p,input,sizeof
input,cudaMemcpyHostToDevice));

        transform<<<1,32>>>(a.p,x.p,y.p);CU(cudaGetLastError());CU(cudaMemcpy(output,
y.p,sizeof output,cudaMemcpyDeviceToHost));

        require(output[0].re==3&&output[0].im==0&&output[1].re==1&&output[1].im==2,"c
omplex transform mismatch");
        std::cout<<"sum=3+0i difference=1+2i\n";
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}

```

Run or inspect

```

nvcc -std=c++17 -O2 main.cu -o example
./example

```

Explanation

Each output row owns one local complex accumulator and reduces across two inputs. Explicit complex multiplication prevents confusing elementwise component multiplication with complex arithmetic.

The sum/difference matrix is the two-point DFT pattern and connects matrix operations to FFT butterflies. Large FFTs should use a validated FFT library rather than scaling this row-per-thread direct transform blindly.

Expected behavior or output

```
sum=3+0i difference=1+2i
```

Concrete experiment

Set both inputs to $1+0i$. Expect $DC=2+0i$ and $difference=0+0i$. Extend the oracle before experimenting with more rows.

Verification status: gcc: target required

Technical references: CUDA-GUIDE, CUDA-MEMORY. See the added references and revision index.

21.2 Building CPU/GPU Signal Pipelines

21.2.1 Overlapping Acquisition and GPU Processing

The engineering question in **Overlapping Acquisition and GPU Processing** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. GPU DSP pipelines must include acquisition, transfer, execution, and result delivery in latency accounting. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements

should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

21.2.2 Measuring End-to-End Latency

The engineering question in **Measuring End-to-End Latency** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. GPU DSP pipelines must include acquisition, transfer, execution, and result delivery in latency accounting. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Preserve acquisition buffers while a prior GPU batch executes

Acquire two synthetic signal blocks in separate pinned slots and measure end-to-end submission-to-completion time.

CUDA target example. No nvcc compiler or NVIDIA GPU was available here; the stated output is the acceptance target, not an observed CUDA run.

cuda_support.hpp

```
#pragma once
#include <cuda_runtime.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
inline void check(cudaError_t e, const char* operation) {
    if(e!=cudaSuccess) throw std::runtime_error(
        std::string(operation)+" : "+cudaGetErrorString(e));
}
#define CU(call) check((call),#call)
template<class T> struct Device {
    T* p=nullptr;
    explicit Device(std::size_t n) {
        if(n) CU(cudaMalloc(reinterpret_cast<void*>(&p),n*sizeof(T)));
    }
    ~Device(){if(p) (void)cudaFree(p);}
    Device(const Device&)=delete;
    Device& operator=(const Device&)=delete;
};
struct Stream {
    cudaStream_t s{};
    Stream(){CU(cudaStreamCreateWithFlags(&s,cudaStreamNonBlocking));}
    ~Stream(){(void)cudaStreamSynchronize(s);(void)cudaStreamDestroy(s);}
    Stream(const Stream&)=delete;
    Stream& operator=(const Stream&)=delete;
```

```

};
struct Event {
    cudaEvent_t e{};
    Event(){CU(cudaEventCreate(&e));}
    ~Event(){(void)cudaEventDestroy(e);}
    Event(const Event&)=delete;
    Event& operator=(const Event&)=delete;
};
template<class T> struct Pinned {
    T* p=nullptr;
    explicit Pinned(std::size_t n)
    {CU(cudaMallocHost(reinterpret_cast<void**>(&p),n*sizeof(T)));}
    ~Pinned(){(void)cudaFreeHost(p);}
    Pinned(const Pinned&)=delete;
    Pinned& operator=(const Pinned&)=delete;
};
inline void require(bool value,const char* message) {
    if(!value) throw std::runtime_error(message);
}
}

```

main.cu

```

#include "cuda_support.hpp"
#include <chrono>
__global__ void power_samples(const float* in,float* out,unsigned n){unsigned
i=blockIdx.x*blockDim.x+threadIdx.x;if(i<n)out[i]=in[i]*in[i];}

int main() {
    try {
        constexpr unsigned n=1024;Pinned<float>
in0(n),in1(n),out0(n),out1(n);
        Device<float> x0(n),x1(n),y0(n),y1(n);Stream s0,s1;
        auto start=std::chrono::steady_clock::now();
        auto submit=[&](float* in,float* out,float* x,float* y,cudaStream_t
s,float amplitude){
            for(unsigned i=0;i<n;++i)in[i]=amplitude;

            CU(cudaMemcpyAsync(x,in,n*sizeof(float),cudaMemcpyHostToDevice,s));
            power_samples<<<4,256,0,s>>>(x,y,n);CU(cudaGetLastError());

            CU(cudaMemcpyAsync(out,y,n*sizeof(float),cudaMemcpyDeviceToHost,s));
        };

        submit(in0.p,out0.p,x0.p,y0.p,s0.s,2);submit(in1.p,out1.p,x1.p,y1.p,s1.s,3);
        CU(cudaStreamSynchronize(s0.s));CU(cudaStreamSynchronize(s1.s));
        auto
elapsed=std::chrono::duration_cast<std::chrono::microseconds>(std::chrono::st
eady_clock::now()-start).count();
        for(unsigned i=0;i<n;++i)require(out0.p[i]==4&&out1.p[i]==9,"power
pipeline mismatch");
        std::cout<<"powers=4,9 end_to_end_us="<<elapsed<<'\n';
    } catch(const std::exception& e) {
        std::cerr << e.what() << '\n'; return 1;
    }
}
}

```

Run or inspect

```
nvcc -std=c++17 -O2 main.cu -o example
./example
```

Explanation

Filling slot1 happens after slot0 has been submitted, so host acquisition work can overlap earlier device work without modifying slot0. Each slot has independent input, output, device storage, and stream ownership.

The steady-clock interval includes synthetic acquisition, transfers, kernel submission, execution, and completion waits, but excludes setup allocations. This is a different scope from CUDA-event kernel time. Actual overlap and latency need target measurements, not a claim based solely on asynchronous API names.

Expected behavior or output

```
powers=4,9 end_to_end_us=<measured integer microseconds>
```

Concrete experiment

Reuse the two slots for a third batch only after its chosen slot completes. Store the acquisition timestamp with each batch and report per-batch completion latency.

Verification status: gcc: target required

Technical references: CUDA-ASYNC. See the added references and revision index.

Worked example — Evidence-driven accelerating signal processing on gpus

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>
```

```

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}

```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only `observed` with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Chapter 22 — Designing a Production Heterogeneous Computing System

This chapter develops **designing a production heterogeneous computing system** as an engineering mechanism rather than an isolated API exercise. The objective is to connect source-level decisions to execution behavior, measurable evidence, and production constraints.

22.1 Designing the Complete CPU/GPU Architecture

22.1.1 Separating Platform, Processing, and Application Layers

The engineering question in **Separating Platform, Processing, and Application Layers** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. A production heterogeneous system needs reference implementations, observability, packaging, and hardware qualification. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while

repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

22.1.2 Defining Clear CPU/GPU Interfaces

The engineering question in **Defining Clear CPU/GPU Interfaces** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. A production heterogeneous system needs reference implementations, observability, packaging, and hardware qualification. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate

the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Define a synchronous processing interface with explicit failure semantics

Keep the application independent of CUDA while requiring a backend to validate shapes and finish outputs before return.

main.cpp

```
#include <span>
#include <vector>
#include <stdexcept>
#include <iostream>

int main() {
    struct Processor{virtual~Processor()=default;virtual void
    apply(std::span<const float>,std::span<float>)=0;};
    struct Cpu final:Processor{void apply(std::span<const
    float>x,std::span<float>y)override{
        if(x.size()!=y.size())throw std::invalid_argument("shape");
        for(std::size_t i=0;i<x.size();++i)y[i]=x[i]+1;
    }};
    auto run=[](Processor& backend)
    {std::vector<float>x{2,4,6},y(3);backend.apply(x,y);return y;};
    Cpu cpu;auto y=run(cpu);
    std::cout<<"application outputs:";for(auto v:y)std::cout<<'
    '<<v;std::cout<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The interface owns no data: spans borrow caller storage for the duration of `apply`. The synchronous contract means a CUDA implementation must establish completion before returning or copy results into separately owned state.

`run` contains application logic but no CUDA types, allocation calls, or device selection. A future-based asynchronous interface would be a deliberate API change

with longer lifetime requirements, not an invisible optimization under this contract.

Expected behavior or output

```
application outputs: 3 5 7
```

Concrete experiment

Implement a fake backend that throws on shape mismatch and verify run propagates the failure. Do not silently substitute zeros for an unavailable GPU.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

22.2 Making the System Testable and Observable

22.2.1 Building CPU Reference Implementations

The engineering question in **Building CPU Reference Implementations** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. A production heterogeneous system needs reference implementations, observability, packaging, and hardware qualification. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements

should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

22.2.2 Verifying GPU Results Against Known Outputs

The engineering question in **Verifying GPU Results Against Known Outputs** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. A production heterogeneous system needs reference implementations, observability, packaging, and hardware qualification. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Report the first CPU-reference mismatch with scale-aware tolerance

Compare independently specified expected outputs against a simulated backend result containing one intentional error.

main.cpp

```
#include <array>
#include <cmath>
#include <iostream>
#include <algorithm>

int main() {
    const std::array<double,4> reference{0,1,1000,-2};
    const std::array<double,4> observed{1e-9,1.0000001,1000.5,-2};
    auto close=[](double a,double b){return
std::isfinite(a)&&std::isfinite(b)&&std::abs(a-
b)<=1e-8+1e-6*std::max(std::abs(a),std::abs(b));};
    std::size_t mismatch=reference.size();
    for(std::size_t i=0;i<reference.size();++i)if(!
close(reference[i],observed[i])){mismatch=i;break;}
    if(mismatch!=2)return 1;
    std::cout<<"mismatch index="<<mismatch<<"
expected="<<reference[mismatch]<<" observed="<<observed[mismatch]<<"'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Absolute tolerance handles values near zero; the relative component grows with signal magnitude. Explicit `isfinite` checks prevent NaN from slipping through a poorly structured comparison.

The 0.5 error at index 2 exceeds the approximately 0.001 tolerance, while the tiny earlier deviations pass. The array is a deliberately injected backend-result fixture, not a measured GPU output. Matching shape, index, expected, and observed values makes a failure actionable.

Expected behavior or output

```
mismatch index=2 expected=1000 observed=1000.5
```

Concrete experiment

Replace the third observed value with infinity and verify it is rejected. Then set it to 1000.0005 and expect no mismatch under the documented tolerance.

Verification status: gcc: passed

Technical references: CPP20. See the added references and revision index.

22.3 Preparing the System for Production

22.3.1 Packaging CUDA and C++ Dependencies

The engineering question in **Packaging CUDA and C++ Dependencies** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. A production heterogeneous system needs reference implementations, observability, packaging, and hardware qualification. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements

should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

22.3.2 Validating Performance Across Hardware Configurations

The engineering question in **Validating Performance Across Hardware Configurations** is not merely whether a program can be made to run. At this level, the useful question is what contract the mechanism establishes, which costs it moves elsewhere, and what evidence would distinguish a correct implementation from one that only succeeds on a convenient input. A production heterogeneous system needs reference implementations, observability, packaging, and hardware qualification. A production design therefore begins by naming the data, ownership, timing, and failure boundaries before choosing an optimization.

For implementation work, separate correctness from performance. First construct a reference path whose behavior is easy to inspect, then introduce the parallel or hardware-specific path behind a narrow interface. Record dimensions, units, allocation ownership, synchronization points, and error returns. This makes the optimized path replaceable and gives tests something stronger than elapsed time to compare. When the implementation depends on CUDA, the CPU reference remains useful even on a machine without a GPU because it defines expected semantics independently of device execution.

Measurement must also match the claim. A kernel duration does not establish end-to-end latency; throughput does not establish deadline behavior; occupancy does not establish useful work; and a numerically close answer is not automatically acceptable unless the tolerance follows from the algorithm and application. Warm-up effects, transfer costs, synchronization, clock behavior, compiler options, device model, and input shape belong in the experiment record. Repeated measurements should be summarized rather than reduced to one unusually favorable observation.

A common failure pattern is to optimize the visible inner loop while leaving the dominant boundary unchanged. Examples include accelerating arithmetic while repeatedly transferring the same data, increasing concurrency while saturating memory bandwidth, or removing a lock while introducing unbounded queues. The corrective method is consistent: trace the complete request or sample path, locate the actual limiting resource, change one mechanism at a time, and rerun both correctness and performance checks.

Verification checkpoint. State the invariant that must remain true, the metric that matters, the environment in which it was measured, and the rollback or fallback path. If any of those four items is missing, the result is still an experiment rather than production evidence.

Package a CUDA target without embedding a workstation path

Provide target-based CMake that selects architectures explicitly and installs one checked CUDA executable; target validation is a separate step from packaging.

Unexecuted CUDA packaging integration: no CUDA compiler or GPU was available in this environment.

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.24)
project(HarborGpu VERSION 1.0 LANGUAGES CXX CUDA)
if(NOT DEFINED CMAKE_CUDA_ARCHITECTURES)
    message(FATAL_ERROR "Set CMAKE_CUDA_ARCHITECTURES for supported deployment GPUs")
endif()
find_package(CUDAToolkit REQUIRED)
add_executable(harbor_gpu main.cu)
set_target_properties(harbor_gpu PROPERTIES CUDA_STANDARD 17
CUDA_STANDARD_REQUIRED YES)
target_link_libraries(harbor_gpu PRIVATE CUDA::cudart)
enable_testing()
add_test(NAME device_smoke COMMAND harbor_gpu)
install(TARGETS harbor_gpu RUNTIME DESTINATION bin)
```

main.cu

```
#include <cuda_runtime.h>
#include <iostream>
int main(){int count=0;auto e=cudaGetDeviceCount(&count);
    if(e!=cudaSuccess||count==0){std::cerr<<"no usable CUDA device\n";return
1;}
    int runtime=0,driver=0;
    if(cudaRuntimeGetVersion(&runtime)!=cudaSuccess||
cudaDriverGetVersion(&driver)!=cudaSuccess)return 2;
    std::cout<<"devices="<<count<<" runtime="<<runtime<<"
```

```
driver="<<driver<<'\\n';  
}
```

validate_target.sh

```
set -eu  
: "${CUDA_ARCH:?Set a CUDA architecture supported by your toolkit and  
deployment GPU}"  
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release  
-DCMAKE_CUDA_ARCHITECTURES="$CUDA_ARCH"  
cmake --build build  
ctest --test-dir build --output-on-failure  
cmake --install build --prefix "$PWD/stage"  
sha256sum stage/bin/harbor_gpu
```

Run or inspect

```
CUDA_ARCH=<supported architecture> sh validate_target.sh
```

Explanation

An imported `CUDA::cudart` target carries toolkit linkage rather than hardcoded `/usr/local` paths. Architecture selection is deployment input, so the build refuses an unspecified target instead of silently choosing a developer machine's assumption.

The executable reports device count and runtime/driver versions as compatibility evidence. It is only a smoke check, not numerical or performance qualification. Preserve a license/dependency inventory and rerun section-specific result and latency tests on each supported hardware class before release.

Expected behavior or output

```
A passing device_smoke test, a devices/runtime/driver line, and a SHA-256  
digest for stage/bin/harbor_gpu; values depend on the target.
```

Concrete experiment

Build for two actually supported deployment architectures, retain separate build directories, and compare artifact size and test results. Do not treat a successful cross-build as execution qualification.

Verification status: gcc: target required

Technical references: CMAKE-BUILDSYSTEM, CUDA-GUIDE. See the added references and revision index.

Worked example — Evidence-driven designing a production heterogeneous computing system

Requirement and design

Build the smallest experiment that preserves the chapter's central contract. Keep input sizes explicit, reject invalid dimensions before launching work, and retain a reference result that can be checked without relying on the optimized path.

Implementation pattern

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

std::vector<float> reference_transform(const std::vector<float>& input) {
    std::vector<float> output(input.size());
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2.0f + 1.0f;
    }
    return output;
}

bool close_enough(float a, float b, float tolerance = 1e-5f) {
    return std::abs(a - b) <= tolerance;
}

int main() {
    const std::vector<float> input{1.0f, 2.0f, 3.0f, 4.0f};
    const auto expected = reference_transform(input);
    const auto observed = reference_transform(input); // replace with
    optimized path

    if (expected.size() != observed.size()) return 2;
    for (std::size_t i = 0; i < expected.size(); ++i) {
        if (!close_enough(expected[i], observed[i])) return 3;
    }
    std::cout << "verified " << observed.size() << " values\n";
}
```

The reference path is deliberately uncomplicated. In a CUDA chapter, replace only observed with the device-backed implementation while preserving the comparison. In a real-time chapter, add bounded timing instrumentation around the complete operation rather than only the arithmetic body.

Evidence and interpretation

A zero exit status establishes agreement for the supplied input and tolerance. It does not establish correctness for every size, numerical range, compiler, GPU architecture, or scheduling condition. Expand tests around empty inputs, odd dimensions, maximum supported sizes, invalid configuration, and repeated execution. For performance work, use an optimized build and separate warm-up from measured iterations.

Failure clinic

If results differ, first determine whether the disagreement is algorithmic, numerical, synchronization-related, or caused by stale data movement. Do not loosen a tolerance until the numerical requirement justifies it. If timing is unstable, inspect allocation, synchronization, queueing, frequency changes, and competing work before attributing the variation to the kernel.

Production engineering deep dive

A high-performance subsystem should be specified with two parallel contracts: a **semantic contract** describing what result is correct and an **operational contract** describing the conditions under which that result must be delivered. The semantic side includes dimensions, units, numerical conventions, permitted error, ordering, and ownership. The operational side includes latency or throughput targets, queue capacity, resource ceilings, startup behavior, shutdown behavior, and the response to overload. Keeping these contracts separate prevents a benchmark from becoming a substitute for correctness.

The data path should be drawn before it is optimized. For a heterogeneous pipeline, identify where bytes originate, which component owns each buffer, when ownership or visibility changes, and where synchronization is required. A useful trace might be acquisition → host staging → transfer → device computation → result transfer → CPU decision → output. Every arrow has a cost and a failure mode. If a buffer is reused before an asynchronous consumer is finished, the bug is a lifetime violation even when the symptom appears as numerical noise.

Capacity planning follows from rates rather than intuition. If an input stage produces work faster than the slowest sustained downstream stage consumes it, a finite queue eventually fills. A larger queue postpones the failure while increasing worst-case latency. Production designs therefore define an overload policy: reject, drop according to an explicit rule, reduce fidelity, spill to a bounded secondary store, or apply backpressure upstream. The correct policy depends on the application, but “allocate until memory runs out” is not a policy.

Numerical verification deserves the same discipline. Floating-point operations may be associated differently on CPU and GPU, and parallel reductions can produce small differences without either implementation being conceptually wrong. The acceptance test should derive tolerances from the numerical problem and expected dynamic range. Use absolute tolerance near zero and relative or mixed tolerances where scale varies. For signal processing, also verify conventions such as FFT normalization, complex sign, bin mapping, windowing, and treatment of partial frames.

Performance experiments should separate **latency**, **throughput**, and **utilization**. Latency measures time for one unit of work through a defined boundary. Throughput measures completed work per unit time under a defined load. Utilization describes how busy a resource appears and is not itself a service outcome. A device can be highly utilized while useful throughput falls because of contention, retries, or inefficient memory access. Conversely, a low-utilization system may be correct when the workload is intentionally bursty.

For CUDA experiments, time the scope you intend to claim. Device events can measure intervals in a stream, while host clocks can measure end-to-end behavior including API and transfer costs. Asynchronous launches require an appropriate completion point before a host-side interval can be interpreted as completed GPU work. Record whether transfers use pageable or pinned memory, whether buffers are reused, and whether the first iteration includes context creation or one-time library initialization.

Resource cleanup must work on failure paths. Device allocations, streams, events, library handles, pinned host buffers, files, and threads all need owners. Wrap resources so destruction releases them and make move operations transfer ownership explicitly. Avoid a cleanup design that assumes every preceding initialization step succeeded. In a long-running service, a small leak per request is eventually a capacity incident.

Observability should answer questions without requiring a debugger on the production machine. Record configuration identity, input shape, queue depth, completed-work counters, bounded latency summaries, error counts, and hardware identity. Avoid logging high-rate per-sample data by default; it can distort timing and exhaust storage. Metrics need units and aggregation rules. A number named latency without milliseconds, microseconds, or a defined percentile is not operational evidence.

Hardware qualification is broader than “the program ran.” Test the supported GPU architectures, driver/runtime combinations, memory capacities, CPU topology, and representative workload shapes. Establish behavior when a requested device is absent, busy, reset, or lacks a required capability. Where CPU fallback is part of the

design, test that fallback directly and determine whether its performance still satisfies the service contract.

Reproducibility requires capturing the toolchain and build. Preserve compiler versions, CUDA toolkit version, CMake configuration, optimization flags, dependency versions, and source revision. Generated code and architecture flags can materially change behavior. A binary copied from an unknown build directory is not a reproducible release artifact merely because its filename contains release.

The release gate should combine evidence. Unit tests establish local behavior, integration tests exercise interfaces, reference comparisons check numerical semantics, sanitizer or analysis tools find classes of defects on supported host paths, stress tests exercise capacity, and performance tests establish measured budgets. No single green check subsumes the others. A production review should state what was tested, what was not tested, and which assumptions remain external.

Operational checklist

Before accepting the chapter mechanism into a larger system, answer these questions:

1. What exact input and output contract is enforced?
2. Who owns every resource, and when does that ownership end?
3. Which operations are asynchronous, and what establishes completion?
4. What happens at maximum supported size and at zero size?
5. How are device/API errors surfaced without losing context?
6. What is the bounded queue or buffering policy?
7. Which numerical tolerance is justified, and by what requirement?
8. Which timing boundary is measured?
9. What hardware and software configuration produced the measurement?
10. What fallback or rollback exists if the optimized path fails?

If these answers cannot be written precisely, the design still contains hidden assumptions. Expose those assumptions before adding more optimization.

Advanced review — from experiment to production evidence

A production review should reconstruct the causal chain from requirement to measurement. Begin with the workload: element count, sample rate, batch shape, arrival pattern, and numerical range. Then identify the implementation configuration: thread geometry, stream count, buffer sizes, algorithm/library selection, and CPU/GPU partition. Finally record the outcome with units. This structure makes a result comparable later; without it, “the optimized version was faster” cannot be reproduced or challenged.

Treat averages cautiously. A real-time or interactive system can satisfy an average while violating important tail deadlines. Preserve distributions or at least representative percentiles and maximum observations over a justified interval. Investigate outliers instead of deleting them because they make a graph untidy. They may expose allocation, page faults, synchronization, thermal behavior, scheduling interference, or periodic maintenance work.

Concurrency also changes the meaning of failure. An error observed by one stream or worker may occur after the initiating caller has continued. Define how failures propagate, whether new work is accepted after a fault, and how in-flight work is drained or cancelled. Shutdown deserves a test of its own: stop accepting work, signal producers and consumers, establish completion, release resources in dependency order, and verify that no background execution refers to destroyed state.

For libraries and kernels, keep interfaces narrower than implementations. Callers should not need to know internal block sizes, scratch-buffer layout, or profiling instrumentation unless those details are genuine configuration contracts. This permits tuning without forcing application changes and makes CPU reference implementations easier to substitute in tests. An optimization that leaks every device detail through the application can increase long-term maintenance cost even when its benchmark improves.

Finally, document negative evidence. If multi-GPU execution, a particular compute capability, hard-real-time scheduling, or a vendor library path was not tested, say so. Engineering confidence comes from matching claims to evidence, not from making the evidence sound broader than it is. The result of this chapter should therefore be both an implementation and an evidence package: source revision, configuration, tests, measurements, limitations, and the decision they support.

Practice and implementation lab

Implement the chapter mechanism behind a narrow interface and keep a CPU reference path. Create at least four cases: ordinary input, a boundary size, invalid input, and repeated execution. Record the compiler, language mode, hardware identity, and build type. For CUDA work, also record device properties and runtime/driver versions.

Acceptance criteria. Correctness checks must pass before timing is interpreted. Errors must be reported at the boundary where they occur. The program must have a defined cleanup path. Any performance claim must name the measured scope and input.

Review reasoning. Explain why a faster inner operation may fail to improve end-to-end latency; why synchronization can be necessary even when it costs time; and why a CPU reference remains valuable in a GPU-focused system.

Chapter takeaway. Optimize the complete contract, not an isolated number.

Appendix A — CUDA and Heterogeneous-Computing Diagnostic Desk Reference

When an optimized path fails, reduce uncertainty in a fixed order. Validate dimensions and configuration before allocating resources. Check every host API result. Check kernel launch status. Establish asynchronous completion at a deliberate boundary. Compare the smallest deterministic input against the CPU reference. Inspect indexing, byte counts, copy direction, and ownership. Only after correctness is localized should profiling guide optimization.

A useful mismatch report contains the first failing index, expected value, observed value, absolute error, relative error where meaningful, input dimensions, and implementation configuration. Avoid dumping entire production arrays merely because a comparison failed. Diagnostics should preserve evidence without becoming a new performance or storage problem.

For latency regressions, decompose the interval into preparation, host-to-device movement, queue delay, device execution, device-to-host movement, CPU post-processing, and delivery. Not every application contains every stage, but every end-to-end claim needs an explicit start and stop boundary.

Appendix B — Real-Time and Streaming Acceptance Worksheet

Define the unit of work and its deadline. Record the admitted arrival rate, queue capacities, overload behavior, allocation policy after steady state begins, synchronization points, and external dependencies that can block progress. Distinguish hard requirements from engineering targets. General-purpose operating systems and ordinary GPU execution do not become hard real-time merely because a benchmark usually finishes quickly.

Exercise startup, steady state, overload, dependency failure, cancellation, and shutdown. Verify that queues remain bounded and resources are reclaimed. A stress test that eventually exhausts memory demonstrates an unbounded design rather than high throughput.

Appendix C — Numerical Verification for Signal Processing

Begin with analytically simple fixtures. An impulse exposes indexing and convolution alignment. A constant sequence exposes gain and DC behavior. A single complex tone exposes frequency-bin mapping and sign conventions. Small hand-computable vectors make correlation and convolution boundaries inspectable. Random inputs are valuable later but are weak first diagnostics because the expected answer is difficult to reason about manually.

For FFT libraries, record transform direction, normalization convention, data layout, batch dimensions, strides, and overwrite behavior. Compare CPU and GPU implementations under the same convention. A constant scaling difference may indicate normalization rather than a broken transform, but it still violates the interface if normalized output was promised.

Appendix D — Production Readiness Review

A heterogeneous component is ready for a release decision when the source and build are reproducible; dependencies are controlled; reference behavior is tested; supported GPU paths are tested on named hardware; numerical tolerances are justified; performance measurements use defined scopes; resource ownership is explicit; queueing is bounded; errors remain observable; and startup, shutdown, and failure recovery have tests.

The review must also list what was not verified. Multi-GPU execution, particular compute capabilities, driver versions, hard-real-time scheduling, or vendor-library configurations should never be implied by tests performed elsewhere. A precise limitation increases the usefulness of evidence because future engineers know which questions remain open.

Appendix E — Glossary

Arithmetic intensity — The amount of computation performed relative to data movement.

Backpressure — A bounded mechanism by which a downstream stage limits upstream production.

Coalescing — Combining memory activity from nearby GPU threads into efficient memory transactions when access patterns permit it.

CPU reference implementation — A deliberately inspectable implementation used to define expected semantics independently of an optimized GPU path.

Deadline — The time by which a unit of work must complete according to a stated requirement.

Device — In CUDA terminology, a GPU or other CUDA-capable execution target controlled by host code.

Divergence — A condition in which threads executing together follow different control-flow paths, potentially reducing execution efficiency.

End-to-end latency — Elapsed time across the complete boundary relevant to the user or system contract, rather than one internal stage.

Heterogeneous computing — Computing that deliberately divides work among processors with different architectures or execution characteristics.

Occupancy — A CUDA resource and scheduling metric describing active warps relative to architectural limits; it is not a direct measure of useful throughput.

Pinned memory — Host memory arranged to support direct asynchronous transfer behavior required by particular CUDA workflows.

Real-time system — A system whose correctness includes timing requirements in addition to logical results.

SIMT — Single instruction, multiple threads; CUDA's execution model in which threads are grouped for hardware execution while retaining thread-level semantics.

Stream — A CUDA ordering domain for asynchronous operations.

Throughput — Completed work per unit time under a stated workload.

Warp — The hardware execution grouping used by NVIDIA GPUs for CUDA threads.

Closing Perspective

Expert C++ engineering is not defined by using the most advanced feature or the fastest processor available. It is defined by making mechanisms explicit enough to reason about them. The CPU, GPU, operating system, compiler, libraries, scheduler, and workload all participate in the observed result. The engineer's task is to preserve correctness across those boundaries while using measurement to decide where complexity is justified.

The progression of this series therefore ends where production work begins: with contracts, evidence, and repeatability. A kernel that is fast but numerically wrong is not an optimization. A pipeline that meets average throughput while growing an unbounded queue is not stable. A real-time claim without a defined deadline and measured distribution is not a timing contract. A build that cannot be reproduced is not a dependable release.

Carry forward the operating discipline used throughout the six volumes: understand the requirement, trace the mechanism, implement the smallest defensible change, verify the result independently, measure the complete path, and document the boundary of the evidence.

Section example revision indexThe original book text and original code are retained. Section-specific examples are independently authored additions for the numbered sections listed below. Their validation records do not certify the retained original manuscript or every optional platform procedure.

- 1.1 Identifying Parallel Work
- 1.2 Understanding Limits to Parallel Speedup
- 2.1 Understanding CPU Optimization Goals
- 2.2 Understanding GPU Optimization Goals
- 3.1 Understanding the CUDA Programming Model
- 3.2 Creating the First GPU Program
- 4.1 Mapping Work to Threads
- 4.2 Organizing Threads into Blocks and Grids
- 5.1 Understanding Warp-Based Execution
- 5.2 Understanding Branch Divergence
- 6.1 Mapping Algorithms to GPU Execution
- 6.2 Managing Kernel Boundaries
- 7.1 Working with Global Device Memory
- 7.2 Understanding Specialized GPU Memory
- 8.1 Using Programmer-Managed Shared Storage
- 8.2 Avoiding Shared-Memory Bottlenecks
- 9.1 Understanding Memory Coalescing
- 9.2 Designing GPU-Friendly Data Layouts
- 10.1 Synchronizing Threads Within Blocks
- 10.2 Coordinating Across Kernels
- 11.1 Understanding Asynchronous GPU Operations
- 11.2 Overlapping Computation and Communication
- 12.1 Detecting Runtime API Errors
- 12.2 Detecting Kernel Execution Failures
- 13.1 Measuring GPU Execution
- 13.2 Locating GPU Bottlenecks
- 14.1 Improving Occupancy
- 14.2 Optimizing Instruction Throughput
- 15.1 Identifying GPU-Appropriate Workloads
- 15.2 Designing Heterogeneous Pipelines
- 16.1 Discovering and Selecting GPU Devices
- 16.2 Distributing Work Across GPUs
- 17.1 Representing Sampled Signals
- 17.2 Implementing Fundamental Signal Operations
- 18.1 Understanding the Discrete Fourier Transform
- 18.2 Applying Fast

Fourier Transforms19.1 Understanding Real-Time Requirements19.2 Reducing
 Timing Variability20.1 Designing Streaming Processing Stages20.2 Maintaining
 Pipeline Throughput21.1 Mapping DSP Operations to CUDA21.2 Building CPU/GPU
 Signal Pipelines22.1 Designing the Complete CPU/GPU Architecture22.2 Making the
 System Testable and Observable22.3 Preparing the System for ProductionTechnical
 references for the section-specific examplesCMAKE-BUILDSYSTEM — CMake: build
 targets and usage requirements[https://cmake.org/cmake/help/latest/manual/
 cmake-buildsystem.7.html](https://cmake.org/cmake/help/latest/manual/cmake-buildsystem.7.html)C++20 — C++20 working draft N4861: language and
 standard-library clauses<https://timsong-cpp.github.io/cppwp/n4861/CUDA-ASYNC>
 — CUDA Programming Guide: asynchronous execution[https://docs.nvidia.com/
 cuda/cuda-programming-guide/02-basics/asynchronous-execution.html](https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/asynchronous-execution.html)CUDA-
 GUIDE — NVIDIA CUDA Programming Guide[https://docs.nvidia.com/cuda/cuda-
 programming-guide/index.html](https://docs.nvidia.com/cuda/cuda-programming-guide/index.html)CUDA-MEMORY — CUDA12.8 Runtime API:
 memory management[https://docs.nvidia.com/cuda/archive/12.8.0/cuda-runtime-
 api/group__CUDART__MEMORY.html](https://docs.nvidia.com/cuda/archive/12.8.0/cuda-runtime-api/group__CUDART__MEMORY.html)CUDA-OCCUPANCY — CUDA12.8 Runtime API:
 occupancy[https://docs.nvidia.com/cuda/archive/12.8.0/cuda-runtime-api/
 group__CUDART__OCCUPANCY.html](https://docs.nvidia.com/cuda/archive/12.8.0/cuda-runtime-api/group__CUDART__OCCUPANCY.html)CUFFT — NVIDIA cuFFT documentation[https://
 docs.nvidia.com/cuda/cufft/index.html](https://docs.nvidia.com/cuda/cufft/index.html)FFTW — FFTW: complex one-dimensional
 transforms[https://www.fftw.org/fftw3_doc/Complex-One_002dDimensional-
 DFTs.html](https://www.fftw.org/fftw3_doc/Complex-One_002dDimensional-DFTs.html)NSIGHT-COMPUTE — NVIDIA Nsight Compute profiling guide[https://
 docs.nvidia.com/nsight-compute/ProfilingGuide/index.html](https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html)